

分类号： \_\_\_\_\_

密级： \_\_\_\_\_

UDC： \_\_\_\_\_

编号： \_\_\_\_\_

专业学位硕士学位论文

# 基于云原生架构的微服务治理方案在电力营销 系统中的应用研究

硕士研究生：朱冠达

指导教师：钱真 副教授

校外导师：迟子悦 高级工程师

学位类别：工程硕士

哈尔滨工程大学

2024年6月

分类号：\_\_\_\_\_

密级：\_\_\_\_\_

UDC：\_\_\_\_\_

编号：\_\_\_\_\_

## 专业学位硕士学位论文

# 基于云原生架构的微服务治理方案在电力营销 系统中的应用研究

(☐产品研发☐工程规划☐工程设计☒应用研究)

硕士研究生：朱冠达

指导教师：钱真 副教授

校外导师：迟子悦 高级工程师

专业类别：计算机技术

所在学院：烟台研究院

论文提交日期：2024年4月22日

论文答辩日期：2024年5月24日

学位授予单位：哈尔滨工程大学

Classified Index:

U.D.C:

A Thesis for the Degree of Master of Engineering

# Research on Application of Microservice Governance Scheme Based on Cloud Native Architecture in Power Marketing System

**Candidate:** Zhu Guanda

**Supervisor:** A.P. Qian Zhen

**Associate Supervisor :** Senior Engineer Chi Ziyue

**Professional category :** Computer Technology

**College :** Yantai Research Institute

**Date of Submission:** April, 2024

**Date of Oral Examination:** May, 2024

**University:** Harbin Engineering University

## 摘要

近年来,随着中国信息化技术的飞速发展以及企业数字化转型的迫切需求,微服务架构以其组件化、模块化的思想逐渐成为主流分布式系统服务治理解决方案,但随着企业应用愈加复杂,微服务架构这种侵入式的服务治理方案也带来了诸多问题,诸如系统升级成本高、服务治理和服务本身紧密耦合、难以对异构微服务进行统一管理等问题。面对上述挑战,本文的主要研究内容分为以下几个方面:

本文以实习企业的电力营销系统为案例,将系统从围绕 Spring Cloud 的侵入式的微服务架构重构为基于服务网格的非侵入式的云原生架构。服务网格以及容器编排工具取代集成服务治理组件 SDK 的方式为整个系统提供服务治理能力。本文选择容器编排工具 Kubernetes 以及服务网格 Istio 作为重构后的基础设施,Istio 会为微服务创建 Envoy 网络代理,这些网络代理即为微服务的 sidecar,这些 Envoy 共同组成了服务网格的数据平面,其对整个系统中的所有服务进行流量拦截和治理,以非入侵的方式实现了服务治理和服务本身的完全解耦。云原生架构可以提高系统的可观察性、安全性、灵活性和可维护性,从而更好地满足复杂分布式系统的服务治理需求。

为了满足以服务网格为基础设施层的微服务通讯需求,本文设计了一个服务通信组件使其更好的适配服务网格,剔除了开源组件中冗余的功能以及 SDK 依赖,使通信组件的 jar 包大小缩小了 4.9%,在通信性能上相较于 Spring Cloud OpenFeign 提高了 3%-6%,同时还保留了原有服务通信组件的编码习惯,保证组件易上手、松耦合、易扩展。

为了解决容器编排工具 Kubernetes 在节点可用网络带宽不足时资源调度不合理的问题,本文提出了基于 AHP-熵权法的多指标资源调度策略 MMRSA,使资源调度器可以感知到节点可用网络带宽以及磁盘 I/O 并依据多个维度的数据进行打分和资源调度,经实验证实,相较于默认资源调度器,本文设计的资源调度器在当前系统环境中的资源调度均衡度提高了 11.3%,能够实现更加均衡的资源调度。

目前,基于 Kubernetes 容器编排工具以及 Istio 服务网格的服务治理方案已经在企业中实际应用,实际运行效果符合企业需求,本套服务治理方案可以帮助企业有效降低微服务的运维成本,提高分布式系统的服务治理能力,为企业最新的基于服务网格的服务治理方案提供有力的技术支持。

关键词：Kubernetes；服务网格；资源调度；云原生

## Abstract

In recent years, there has been a rapid development of information technology in China. An increasing number of enterprises necessitate digital transformation. Microservices architecture is gradually emerging as a mainstream solution for service governance in distributed systems, owing to its characteristic modularity and componentization. However, as enterprise applications become increasingly complex. Deploying service governance through microservices architecture also presents numerous challenges. For instance, challenges include high system upgrade costs, tight coupling between service governance and the services themselves, and difficulty in uniformly managing heterogeneous microservices. Faced with the aforementioned challenges, this thesis's primary research focuses on the following aspects:

This thesis utilizes the electric power marketing system of a practicum enterprise as a case study. Refactoring the system from an invasive microservices architecture centered around Spring Cloud to a non-invasive cloud-native architecture based on service mesh. Service mesh and container orchestration tools replace the integrated service governance component SDK, providing service governance capabilities for the entire system. The refactored infrastructure in this thesis is based on Kubernetes as the container orchestration tool and Istio as the service mesh. Istio creates Envoy network proxies for microservices. These network proxies serve as sidecars for the microservices. These Envoy proxies collectively form the data plane of the service mesh. The data plane intercepts and manages traffic for all services in the system, achieving complete decoupling of service governance from the services themselves in a non-intrusive manner. Cloud-native architecture can enhance system observability, security, flexibility, and maintainability, Therefore, it can better satisfy the requirements of complex distributed systems.

To meet the communication requirements of services based on the service mesh infrastructure, a service communication component is designed to better adapt to service mesh environments.. This component eliminates redundant functionalities and SDK dependencies from open-source components, resulting in a 5% reduction in the size of the communication component's JAR file. This component achieves a 3%-6% improvement in communication performance compared to Spring Cloud OpenFeign. This component retains the coding con-

ventions of the original service communication component. The component ensures ease of use, loose coupling, and scalability.

To solve the problem of Kubernetes resource scheduling inefficiency due to insufficient network bandwidth on nodes. The multi-index resource scheduling strategy MMRSA based on AHP-entropy method is proposed in this thesis. This algorithm enables the resource scheduler to perceive the available network bandwidth and disk I/O of nodes and allocate resources based on scoring and scheduling across multiple dimensions of data. The experimental results indicate that compared to the default resource scheduler, the resource scheduler designed in this thesis achieves more balanced resource allocation.

Currently, service governance solutions based on Kubernetes container orchestration tools and Istio service mesh have been practically applied in enterprises, meeting user requirements effectively. This suite of service governance solutions can help enterprises reduce the operation and maintenance costs of microservices, improve the efficiency of service governance in distributed systems, and provide robust technical support for the latest service governance solutions based on the service mesh.

**Key words:** Kubernetes, Service mesh, Resource scheduling, Cloud-native

# 目 录

|                                      |    |
|--------------------------------------|----|
| 第 1 章 绪论.....                        | 1  |
| 1.1 研究背景和意义.....                     | 1  |
| 1.2 服务网格研究现状.....                    | 2  |
| 1.3 资源调度算法研究现状.....                  | 4  |
| 1.4 本文主要研究工作.....                    | 5  |
| 1.5 论文的组织架构.....                     | 6  |
| 第 2 章 云原生架构相关技术 .....                | 9  |
| 2.1 云原生技术.....                       | 9  |
| 2.1.1 Docker 容器化技术 .....             | 9  |
| 2.1.2 Kubernetes 容器编排技术 .....        | 10 |
| 2.1.3 Service Mesh 服务网格技术.....       | 13 |
| 2.2 服务治理技术.....                      | 14 |
| 2.2.1 侵入式的服务治理技术.....                | 15 |
| 2.2.2 非侵入式的服务治理技术.....               | 16 |
| 2.3 本章小结.....                        | 18 |
| 第 3 章 云原生架构服务治理方案设计与应用研究 .....       | 19 |
| 3.1 服务治理方案设计与分析 .....                | 19 |
| 3.1.1 网关与集群流量下发治理方案.....             | 19 |
| 3.1.2 集群内服务治理方案.....                 | 24 |
| 3.2 服务通信组件的需求分析与设计 .....             | 25 |
| 3.2.1 服务通信组件的需求分析.....               | 26 |
| 3.2.2 服务通信组件的总体设计.....               | 28 |
| 3.2.3 服务通信组件的详细设计.....               | 30 |
| 3.3 基于 AHP-熵权法的资源调度优化算法 MMRSA .....  | 33 |
| 3.3.1 Kubernetes 默认资源调度算法及问题分析 ..... | 33 |
| 3.3.2 AHP 权重分析法 .....                | 34 |
| 3.3.3 熵权法权重分析法.....                  | 36 |
| 3.3.4 资源调度优化算法 MMRSA.....            | 38 |
| 3.4 基于 MMRSA 算法的优化资源调度器需求分析与设计 ..... | 40 |
| 3.4.1 调度器需求分析.....                   | 40 |
| 3.4.2 调度器方案设计.....                   | 40 |
| 3.4.3 调度器资源指标获取方案.....               | 42 |
| 3.4.4 调度器详细设计.....                   | 43 |
| 3.5 本章小结.....                        | 44 |



|                              |    |
|------------------------------|----|
| 第 4 章 电力营销系统构建与实验分析 .....    | 45 |
| 4.1 服务网格基础设施构建.....          | 45 |
| 4.2 云原生架构系统构建与相关优化分析测试 ..... | 46 |
| 4.3 服务通信组件的测试.....           | 53 |
| 4.3.1 实验目标.....              | 53 |
| 4.3.2 实验方案.....              | 53 |
| 4.3.3 实验结果分析.....            | 54 |
| 4.4 MMRSA 算法资源调度能力测试.....    | 55 |
| 4.4.1 实验目标.....              | 55 |
| 4.4.2 实验方案.....              | 55 |
| 4.4.3 评价指标.....              | 56 |
| 4.4.4 实验结果分析.....            | 56 |
| 4.5 本章小结.....                | 57 |
| 结论.....                      | 59 |
| 参考文献.....                    | 61 |

## 第 1 章 绪论

### 1.1 研究背景和意义

近年来，中国的互联网行业飞速发展，在信息化技术提高了人们生活质量的同时，人们也对系统软件产品提出了更高的要求<sup>[1]</sup>。系统规模的快速扩张以及产品需求的不断增加给企业带来了巨大的挑战，同时也推动着系统架构的不断演进。

最早的系统服务通常采用单体架构<sup>[2]</sup>。这种架构简单直观，但存在可伸缩性受限、复杂度高、技术栈受限等问题。2014 年微服务这一概念的出现为系统服务治理带来了新的解决方案<sup>[3]</sup>。微服务架构进一步拆分服务，使各个服务更加小型且可独立部署。每个微服务都是一个独立的模块，并使用通信协议进行交互<sup>[4]</sup>。微服务治理技术在这个阶段百花齐放，以 java 语言为例，各个公司都在推出不同的微服务治理的中间件，例如 Nacos、Eureka、Sentinel、Seata 等等<sup>[5]</sup>。微服务架构比单体架构耦合度更低，具有很强的伸缩性和灵活性，但同样也存在很多问题，比如要实现分布式的微服务治理，需要通过集成 SDK 的方式引入各种中间件来保证服务的稳定与可靠，这种服务治理的方式带来的问题就是使得业务代码和服务治理代码耦合度很高，服务治理代码需要侵入到业务代码当中来完成治理<sup>[6]</sup>。此外，这些服务治理中间件还会限制项目的开发语言、导致中间件版本碎片化，在中间件升级或者模块依赖升级时，可能会因为繁杂的中间件版本匹配导致难以预料的问题<sup>[7]</sup>。

为了解决以上问题，服务网格技术应运而生，如图 1.1 是云原生架构的核心技术体系，云原生架构是微服务架构的进一步演进，它着重于在服务之外解决微服务之间的通信和治理问题。服务网格通过引入一个专门的基础设施层来处理通信，这样就可以避免侵入式的服务治理，同时提供了更多的控制能力和可观察性<sup>[8]</sup>。在服务网格当中，每一个微服务模块都有自己的一个代理服务，每个模块与其他模块或外部服务的通信都需要经过自己的代理服务，所以代理服务可以监控到所有打到对应微服务模块的流量并加以治理，这些代理服务实际上是受服务网格控制面控制的，代理服务们组成了服务网格的数据面，服务网格通过配置下发的方式来控制网格中代理服务的服务治理规则<sup>[9]</sup>。这样，各模块在开发的时候，就不需要再考虑服务治理中间件带来的问题了，开发语言也不受限制变得多样化，也适应了当前大型系统当中多模块语言复杂、难以统一治理的问题<sup>[10]</sup>。

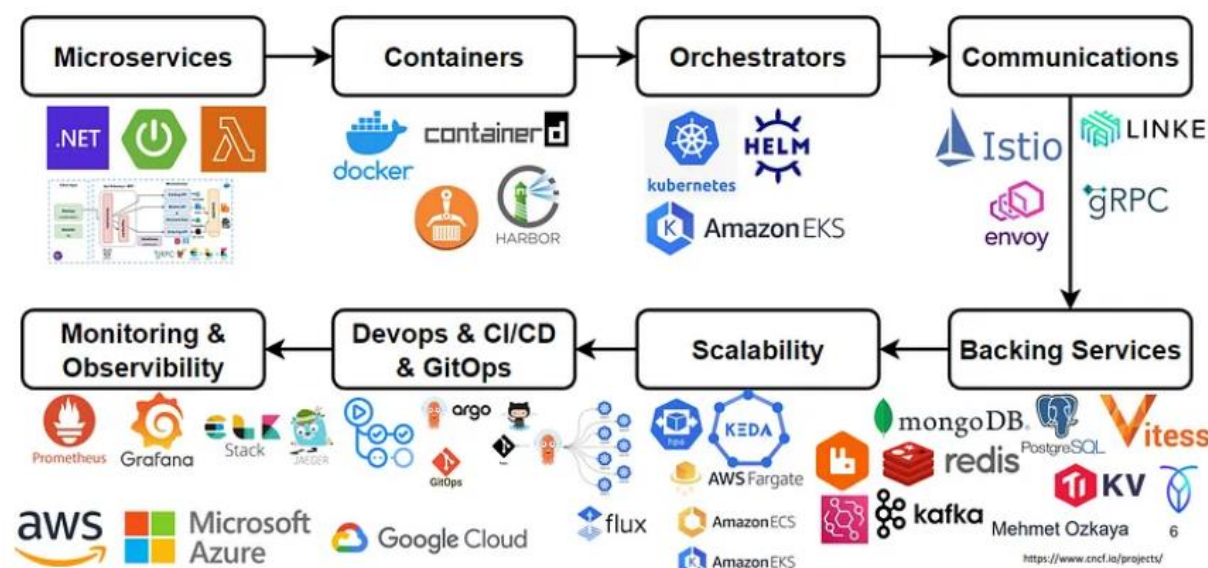


图 1.1 云原生架构技术体系

本课题的研究基于企业实际的电力营销系统项目，研究基于服务网格的微服务治理方案可以提供新的服务治理框架和思路，在保证更高服务稳定性的同时，为企业降本增效，为项目迭代更新、开发运维提供便利。

## 1.2 服务网格研究现状

服务网格（Service Mesh）这一概念最早在 2016 年被 Buoyant 公司提出，该公司开发了 Linkerd，Linkerd 最初是为了解决 Twitter 的微服务架构中的网络问题而开发的<sup>[11]</sup>。随着时间的推移，Linkerd 逐渐发展成为一个开源项目并持续改进和增强其功能，包括改进性能、增加安全性、提供更好的监控和管理功能等。在 Linkerd 2.0 版本中，Buoyant 重新设计了 Linkerd 的架构，引入了基于 Rust 的新代理，即 Linkerd2-proxy，并采用了面向数据平面的设计。这使得 Linkerd 具有更好的性能、可扩展性和稳定性。

在服务网格这一概念引起关注之后，有很多公司对服务网格进行了实践探索，例如在 2017 年首次发布的由 Google、IBM 和 Lyft 共同开发的 Istio<sup>[12]</sup>。Istio 在首次发布后受到了广泛关注，并以其架构简单、丰富的社区支持和生态、通用性强以及高度适配的监控系统等特点迅速成为了服务网格领域的领导者之一。Istio 逐渐成为现行最流行、使用最广泛的服务网格解决方案<sup>[13]</sup>。它的设计理念和功能非常丰富，包括流量管理、安全性、策略实施、监控和追踪等方面<sup>[14]</sup>。随着时间的推移，Istio 持续改进和增强其功能，加强了与 Kubernetes 等容器编排平台的集成，提高了性能和可扩展性，并加强了与云原生生态系统的合作和整合<sup>[15]</sup>。使用 Istio 可以大大提高项目开发运维的便利性和高效性。

除了 Linked 和 Istio, 很多国内的互联网公司也都在进行自己的服务网格技术建设, 例如蚂蚁金服的 SOFA Mesh、华为云的 CSE Mesher、腾讯的 Tars Mesh 等<sup>[16]</sup>。SOFA Mesh 自研了 MOSN 替代 Istio 中的 Envoy 作为服务网格的数据平面解决方案。华为也基于 Istio 改进开发出了 ASM 等云原生技术产品。

吴封斌等<sup>[17]</sup>详细探讨了微服务架构下服务治理的升级方案。它从传统微服务治理的挑战出发, 介绍了服务网格的概念以及 Istio 的架构和功能。文章通过搭建基于 Istio 的服务网格云原生架构示例, 展示了服务网格如何有效地管理微服务之间的流量、实现策略和遥测、提供可视化工具以及加强系统安全性。证明了服务网格能够完全取代微服务架构成为服务治理的新一代解决方案。

Theodoropoulos 等<sup>[18]</sup>详细讨论了云原生架构的特点, 如分布式架构和动态可伸缩性, 并指出这些特点导致的特定的安全挑战。文章还探讨了如何通过零信任架构、服务网格、安全开发生命周期来加强云原生服务的安全性。

Laurentius N 等<sup>[19]</sup>比较了不同服务网格代理服务器应用的响应时间和硬件资源利用率, 评估了常见微服务范式的性能, 使高请求率的企业应用能够以最低的时延和高效率的资源服务于客户端。结果表明, Istio 入口代理适用于请求率较高的微服务应用, 而 nginx 入口代理在处理较低请求时表现较好。

李海飞等<sup>[20]</sup>针对微服务系统中存在的级联故障问题提出了一套级联故障预测方法。其通过在 Sidecar 中嵌入故障预测组件, 对进出系统的所有流量进行检测。当有大量请求进入系统时, 该组件被唤醒, 并针对当前请求的调用关系与资源依赖关系构建模型, 通过 GRU 神经网络实现故障预警。他们提出了微服务健康的概念 (类似服务质量的概念), 以更准确地评估微服务系统是否出现异常。通过实验平台验证了方法的有效性。证明了使用服务网格作为服务治理的基础设施时的便利和高效性。

宋美娜等<sup>[21]</sup>基于开源的服务网格框架 Istio 和容器编排框架 Kubernetes 设计于实现了一个调用链追踪系统。对比于使用 SpringCloud 框架要对原系统嵌入的源代码进行大量修改, 修改成本较高。通过 Sidecar 的方式避免对原系统嵌入的源代码进行修改, 降低了跟踪系统与原系统之间的代码耦合。

对近几年的微服务治理发展历程进行总结分析, 可以发现云原生技术正在深刻的革新互联网技术领域, 云原生技术可以让开发者更加轻松的构建和管理应用, 应用部署上云后, 通过云原生技术来保证服务的高可用。云原生致力于让开发者更关注业务, 将所有的服务治理工作交给云平台的基础设施层, 实现服务治理逻辑和业务逻辑的完全解耦。鉴于以上服务网格的优点, 本文基于实习企业的电力营销系统, 将基于微服

务架构的服务治理能力迁移到服务网格当中，对其服务治理方式进行应用研究。

### 1.3 资源调度算法研究现状

Kubernetes 的资源调度算法应用于 Kubernetes 资源调度器，资源调度器 Kube-Scheduler 通过特定的调度算法决定将需要调度的资源分配到哪个服务器节点上。调度算法的好坏直接决定了 Kubernetes 集群的各项资源的负载情况以及集群的稳定性。

Townend 等<sup>[22]</sup>提出了一种针对 Kubernetes 的全面调度系统，该系统取代了默认的调度器，并同时考虑软件与硬件模型，以提高数据处理的效率。通过在基于软件的解决方案中引入硬件建模，该智能调度器在数据处理效率上提升显著，资源消耗量减少了 10-20%，但是该算法并未在资源调度均衡度方面做出优化。

Buyya R 等<sup>[23]</sup>提出了一种基于 Kubernetes 系统的弹性容器供应的反馈控制方法。这种方法结合了线性模型，以提高输出误差的准确性。作者将他们的方法与现有的几种算法在真实的 Kubernetes 集群上进行了比较，发现它可以明显的降低成本开销，但是该调度算法并未在资源调度打分因素层面做出改进。

Zhong Z 等<sup>[24]</sup>提出了一种用于 Kubernetes 云计算基础设施中的成本效益容器编排异构任务分配策略，这种策略支持弹性计算资源。所提出的策略具有三个主要特点：支持异构作业、通过自动缩放算法调整集群大小、以及拥有重调度机制，用于关闭利用率低下的虚拟机实例并在不丢失任务进度的情况下重新分配相关作业。与默认的 Kubernetes 框架相比，它可以将总成本降低 23-32%，但是该调度算法并未对静态的调度策略做出改进。

Ogbuachi 等<sup>[25]</sup>提出了一种改进的 Kubernetes 调度算法，除了考虑软件状态外，还考虑了物理硬件状态和网络参数，以便更好地协调和管理边缘计算应用程序。他们比较了所提出的调度算法与默认的 Kubernetes 调度算法，结果表明，改进的调度算法提供了更好的容错性和动态编排能力，该调度算法虽然引入了更多的打分因素，但是其算法权重的分配过于简单不够合理。

Song S 等<sup>[26]</sup>提出了一个基于拓扑的 GPU 调度框架用于 Kubernetes。该框架基于传统的 Kubernetes GPU 调度算法，但引入了 GPU 集群拓扑的概念，该拓扑通过 GPU 集群资源访问成本树来恢复。这使得可以更有效地调度不同 GPU 资源应用场景。提出的框架已在腾讯的生产实践中使用，该算法提高了 GPU 集群的资源利用率约 10%，该算法在进行资源调度的时候考虑了集群的 GPU 资源，但是这个调度策略并不适合更关注磁盘和网络带宽的系统集群。

Wei Y 等<sup>[27]</sup>提出了两种动态调度算法，平衡磁盘 I/O 优先级和平衡 CPU-磁盘 I/O 优先级，以解决 Kubernetes 调度器未考虑节点磁盘 I/O 负载的问题。平衡磁盘 I/O 优先级旨在改善节点间的磁盘 I/O 平衡，而平衡 CPU-磁盘 I/O 优先级旨在解决单个节点上 CPU 和磁盘 I/O 负载不平衡的问题。实验证明，两种动态调度算法比 Kubernetes 默认的调度算法更有效，作者考虑到了磁盘 I/O 瓶颈对系统的影响，但是在打分因素中并未包含网络带宽，这对于数据流转量大的系统来说并不合理。

人们针对不同的应用场景，对不同的资源调度算法进行了探索，本电力营销系统当中存在数据采集模块并且各模块之间数据流转也比较频繁，会导致部分模块对网络带宽和磁盘 I/O 的要求会比较高，基于当前系统现状，为了实现更加均衡的资源调度，本文对资源调度算法进行研究，设计了一个基于多指标的资源调度算法，研究各指标的得分计算方式以及更加合理的权重分配方案，以实现更加均衡的资源调度，避免某个服务器节点负载过高，提高系统稳定性。

## 1.4 本文主要研究工作

本文的主要研究内容如图 1.2 所示，本文基于实习企业的“电力营销技术支持系统”项目，针对企业实际遇到的系统服务治理问题，研究基于云原生架构的微服务治理方案，本文将从以下几个方面开展研究：

（1）为了解决在微服务治理中遇到的例如中间件版本碎片化、组件升级困难、系统监控可视性差、服务治理和业务代码耦合度高等问题，本文研究基于云原生架构的微服务治理方案，研究服务网格如何为集群内的系统服务提供服务注册发现、熔断、限流、降级等服务治理能力。研究网关的重构方案、集群流量下发方式以及服务间的通信方式，研究如何剥离微服务架构中的服务治理中间件，将服务治理任务全部交给服务网格来完成。

（2）为了满足服务网格中的服务间通信需求，研究设计一款服务通信组件，研究如何剥离现有开源框架中对于服务网格架构冗余的通信辅助功能，研究如何使用更少的依赖库实现更轻量化的服务间通信，研究保留开源组件编程习惯、支持多环境配置以及链路追踪的方式。

（3）为了解决服务部署过程中 Kubernetes 集群资源调度不均衡的问题，研究设计一个资源调度算法来实现更加均衡的资源调度。研究资源调度算法中节点资源的打分方法以及打分因素的权重计算方法，研究集群监控中心的集成方式与资源调度算法的数据获取方法。

(4) 从各业务模块轻量化程度、系统多语言可扩展性、集群监控支持、流量管理灵活性等方面对重构后的系统进行优化分析。然后对本文实现的定制服务通信组件进行功能性和非功能性测试。最后对本文提出的基于 AHP-熵权法的多指标资源调度算法进行资源调度均衡度测试与分析。

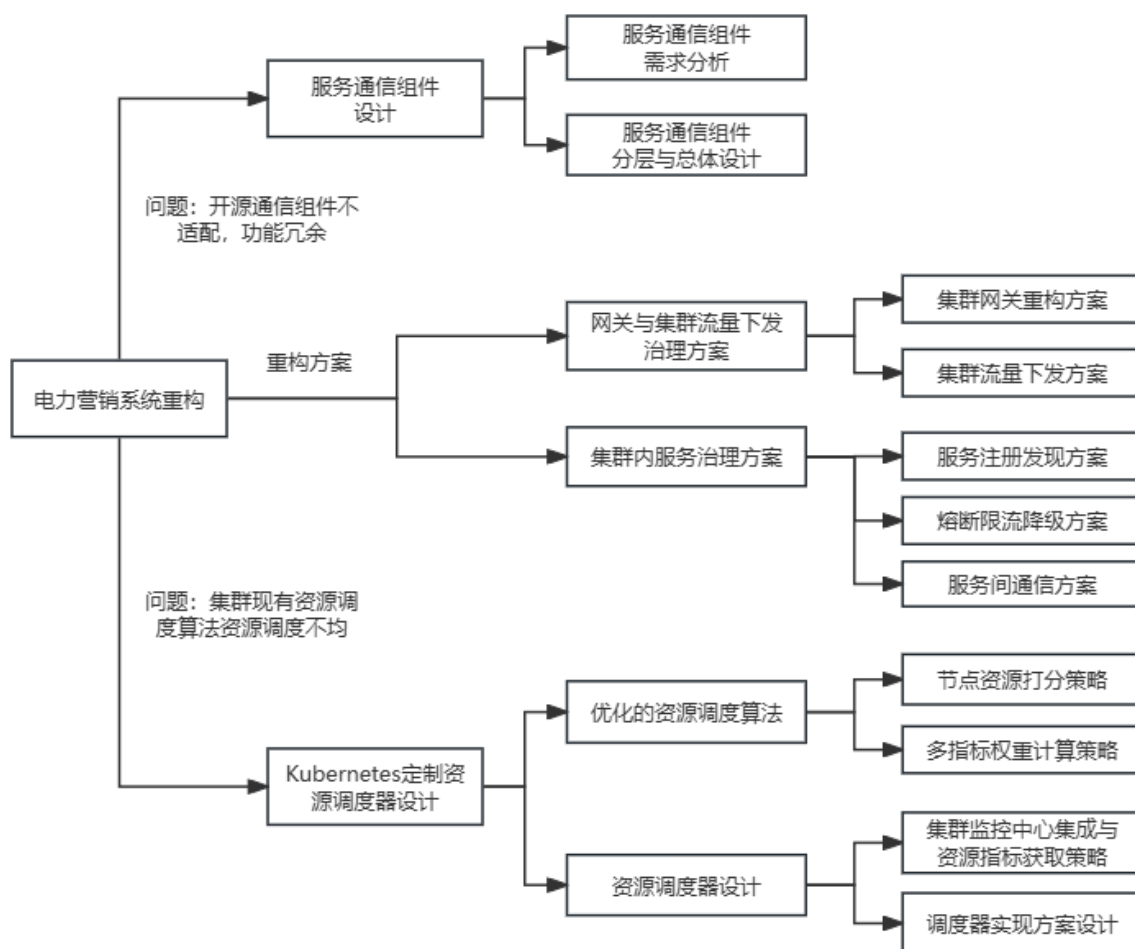


图 1.2 论文主要研究内容

## 1.5 论文的组织架构

第一章为绪论，主要介绍了服务网格技术以及云资源调度策略的研究意义和背景，并对国内外研究现状进行了概述。

第二章介绍了与本文研究相关的技术基础，包括 Docker、Kubernetes、服务网格等云原生技术以及两种服务治理技术，为后续系统的重构提供理论基础支持。

第三章对项目架构从微服务架构到服务网格架构的重构方案进行设计，其中包括对系统中网络流量的重新规划以及现有微服务治理组件的剥离与升级方案设计。针对

现有开源远程通信组件不够适配 Istio 服务网格的现状，本文设计了一款轻量级的服务通信组件，本文对该通信组件进行了总体以及分层设计，以达到足够适配 Istio 服务网格、足够轻量化以及兼容开源组件编码习惯、方便组件迁移的目的。本文还提出了基于 AHP-熵权法的多指标资源调度策略 MMRSA，该调度算法在进行资源调度的时候，会将候选服务器节点的 CPU、内存、网络带宽以及磁盘 I/O 利用率作为节点优选阶段打分环节的重要参考依据，并对不同因素的权重分配方式进行了优化，可以有效提供更加均衡的资源调度服务。

第四章从各业务模块轻量化程度、系统多语言可扩展性、集群监控支持、流量管理灵活性等方面对重构后的系统进行优化分析。对远程通信组件的功能性和非功能性进行测试。同时对基于 MMRSA 算法的定制调度器的资源调度均衡度进行测试，验证了该调度器在实际应用场景中资源调度均衡度的优化效果。





## 第 2 章 云原生架构相关技术

本章主要介绍本文使用到的一些关键技术，首先是对 Docker，Kubernetes 和 Istio 服务网格技术进行介绍，其次分析侵入式开发方式的微服务架构和非入侵式开发方式的服务网格云原生架构的差异以及云原生架构的优势，为后文的服务治理方案设计奠定基础。

### 2.1 云原生技术

Docker 容器化技术是云原生技术的根基，Kubernetes 容器化技术与 Istio 服务网格技术都是在 Docker 容器化的基础之上衍生出来的技术，Docker 用于容器化应用程序，创建独立、可移植的运行环境，Kubernetes 用于自动化管理和编排这些容器，提供强大的集群管理功能，服务网格则构建在 Kubernetes 之上，为整个系统集群提供更加灵活的流量管理能力、更强大的集群监控能力和服务治理能力，使得整个系统更易于扩展、部署和维护<sup>[28]</sup>。

#### 2.1.1 Docker 容器化技术

Docker 是一种开源工具，用于轻松打包、分发和以容器的形式运行应用程序及其所有依赖项。Docker 的核心概念是容器化，即将应用程序和其相关组件打包到一个独立、可移植的容器中<sup>[29]</sup>。这个容器包含运行应用所需的所有文件、库和设置，使得应用能够在不同环境中保持一致性，不会受到底层操作系统或其他软件的影响，减少了由于环境变化而导致的问题。

如图 2.1 所示，Docker 使用操作系统层的虚拟化技术，称为容器技术，以在主机系统上创建和运行容器。每个容器都是一个独立的、轻量级的执行单元，共享主机操作系统的内核<sup>[30]</sup>。这种轻量级的虚拟化与传统的虚拟机的资源隔离方式相比，可以使容器能够快速启动、占用更少的资源，并在不同环境中提供一致的运行时环境。

容器化技术有诸多优点，如 Docker 容器可以在各种不同的操作系统上运行，包括 Linux、Windows 和 macOS，还为开发者提供了跨平台的灵活性，它在占用更少资源的同时保证资源隔离性。Docker 还消除了开发、测试和生产环境之间的不一致性，确保应用程序在整个生命周期中的一致性表现。而且 Docker 的镜像构建过程相对快速，这使得开发者能够更快速地构建和交付应用程序。

总体而言, Docker 的出现极大地简化了应用程序的构建、部署和管理过程, 提高了开发者和运维人员的效率, 使得应用程序更加灵活、可移植和可靠<sup>[31]</sup>。Docker 成为现代软件开发和部署的重要工具之一。

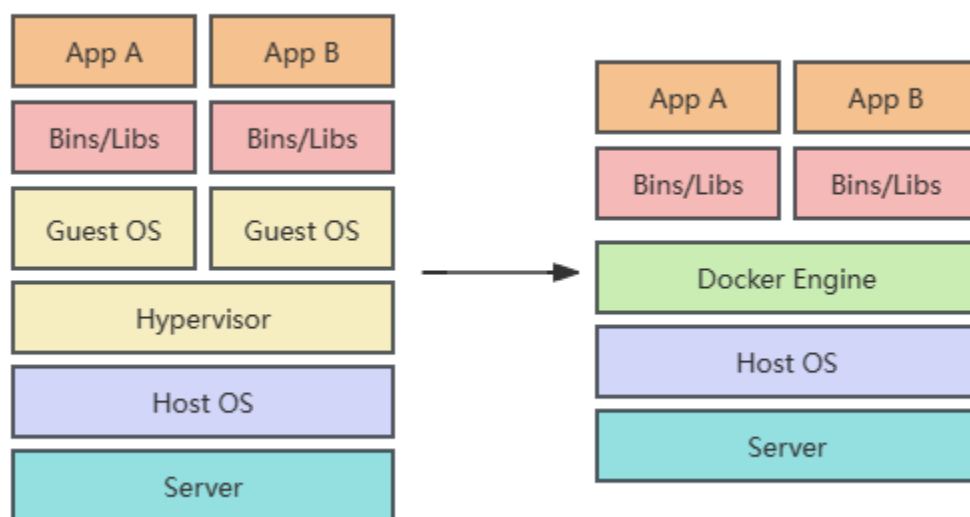


图 2.1 虚拟化技术演进示意图

### 2.1.2 Kubernetes 容器编排技术

Kubernetes 是一个开源的容器编排平台, 用于自动化、部署、伸缩和运维容器化应用程序<sup>[32]</sup>。它提供了一种强大的方式来管理容器化应用程序的生命周期, 包括部署、伸缩、负载均衡、自愈和滚动更新等功能。Kubernetes 最初由 Google 开发, 并于 2014 年开源, 现在由云原生计算基金会 CNCF 维护。

Kubernetes 的优势体现在多个方面, 作为容器编排工具, Kubernetes 解决了容器编排的问题, 使得开发者能够轻松地管理和编排分布式应用程序中的多个容器。在容器部署方面, Kubernetes 提供了自动化的应用程序部署和伸缩机制, 使得开发者能够更容易地将应用程序部署到生产环境, 并根据需要动态调整应用程序的规模。为了保证所管理系统的稳定运行, Kubernetes 设计了高可用的架构, 确保应用程序在面临节点故障时能够保持可用。它支持自动恢复和负载均衡, 提高了应用程序的可靠性<sup>[33]</sup>。在请求路由方面, Kubernetes 具有内置的服务发现机制, 允许应用程序中的不同服务相互发现和通信。同时, 它提供负载均衡功能, 确保流量被均匀分发到可用的服务实例上。Kubernetes 还支持滚动更新, 允许在不中断服务的情况下逐步更新应用程序。如果更新导致问题, 可以轻松地回滚到之前的版本。在进行资源管理时, Kubernetes 允许对容器

和节点的资源进行管理和优化，确保资源被有效地利用。它可以自动调整容器的资源分配，以适应应用程序的需求<sup>[34]</sup>。

以上就是 Kubernetes 的优势，下面介绍 Kubernetes 知识框架中重要的资源对象，为后续的论文工作做铺垫，如图 2.2 是 Kubernetes 中核心资源对象的关系图。

**Pod:** Kubernetes 中最小的可部署单元，可以包含一个或多个容器。Pod 共享相同的网络命名空间、存储卷和 IP 地址，它们一起运行在同一个 Node 上。Pod 通常用于将相关的容器组合在一起，共享相同的上下文和资源。

**ReplicaSet:** 用于确保在任何时候都有指定数量的 Pod 副本在运行。它允许进行水平扩展，根据需要自动增加或减少 Pod 的数量。

**Deployment:** Kubernetes 中的高级控制器，用于定义和管理 Pod 副本的部署。它支持滚动更新、回滚、扩展和升级应用程序。

**Service:** Service 定义了一组 Pod，并提供了一个统一的入口，使得应用程序的不同组件可以相互发现和通信。它实现了负载均衡，确保流量被均匀分发到后端的 Pod 上。

**Ingress:** 用于定义将外部流量引导到集群内服务的规则。它提供了一种灵活的方式来管理外部访问，并支持 SSL 终止、负载均衡和路由规则。通过使用 Ingress，可以在集群外部访问集群中的服务，而无需直接暴露服务的 NodePort 或 LoadBalancer。

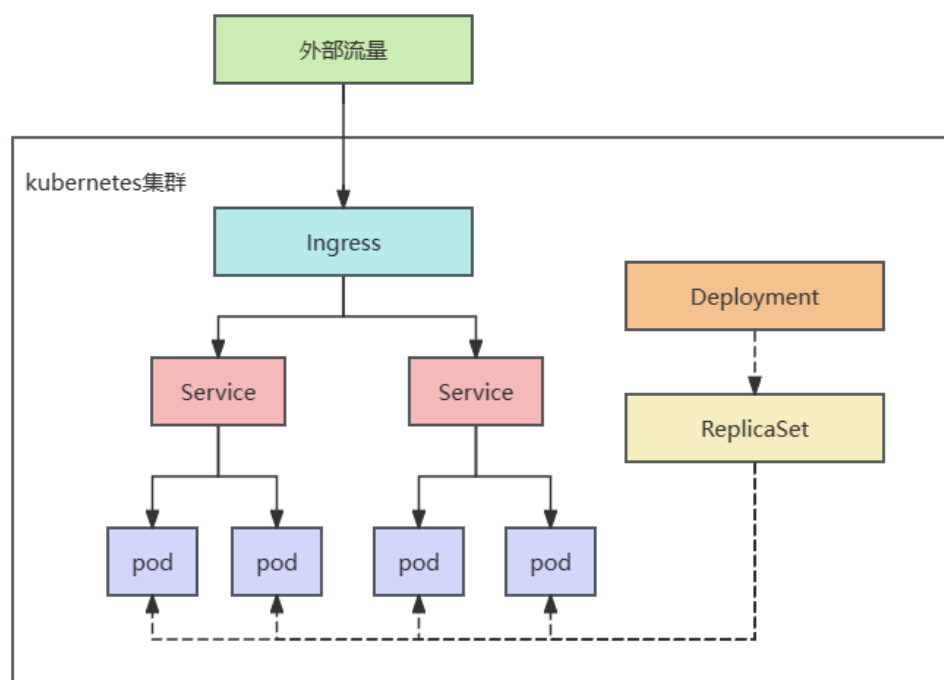


图 2.2 Kubernetes 中核心资源对象关系图

**API Server:** Kubernetes 系统的入口，接收用户和其他组件的请求，并负责对集群状态的操作。所有的控制命令都通过 API Server 进行处理。

**Controller Manager:** Controller Manager 包含多个控制器，负责监控集群中各种对象的状态，并根据定义的期望状态调整实际状态，保持集群处于用户指定的状态。

**Kubelet:** 运行在每个 Node 上的代理，负责维护 Pod 的生命周期，与运行时的容器进行通信，并报告节点的状态给 Master 节点。

Kube Proxy: 负责维护 Node 上的网络规则, 为 Pod 提供网络代理服务。它支持 Service 的负载均衡, 实现了 Pod 之间的网络通信。



### 2.1.3 Service Mesh 服务网格技术

在如今的云原生领域中，Kubernetes 已经成为管理和编排容器化应用的事实标准<sup>[36]</sup>。然而，随着微服务架构的普及，服务之间的通信变得越来越复杂，这引发了新的挑战。为了更好地解决这些挑战，Service Mesh 应运而生，成为云原生架构中不可或缺的一部分。

Service Mesh 是一种专注于处理服务之间通信的独立基础设施层<sup>[37]</sup>。它通过在应用程序服务之间注入轻量级的代理（sidecar），提供了对服务发现、负载均衡、安全性、监控等关键功能的抽象和集中控制。通过这些功能从应用程序中解耦出来，Service Mesh 简化了开发者的任务，同时提高了整个系统的可观察性和可维护性。

引入 Service Mesh 的一个关键优势是透明的服务间通信<sup>[38]</sup>。通过将通信逻辑移至代理层，Service Mesh 能够提供一致性的服务发现和负载均衡，确保了微服务之间的无缝通信。此外，它还增强了安全性，通过提供流量加密、认证和授权等功能，保障了通信的隐私和完整性。

在众多 Service Mesh 实现中，Istio 脱颖而出，成为领先的选择。作为一个开源项目，Istio 建立在 Envoy 代理之上，提供了强大的功能集合，使得微服务架构更加灵活和可控，如图 2.4 是 Istio 的架构图。Istio 的架构涉及多个组件，主要分为数据平面和控制平面。数据平面是 Istio 中处理实际网络流量的组件，它位于应用程序和外部服务之间，并负责拦截、处理和路由所有的网络流量。控制平面是 Istio 的集中管理和配置中心，负责定义和分发网络策略、配置和规则。控制平面由 Pilot、Citadel、Galley 组成，数据平面主要依赖部署在 Pod 上面的 Envoy 来提供，以下是这些组件以及它们的作用：

**Envoy:** 它是 Istio 的数据平面代理，负责处理服务之间的所有网络通信。每个应用程序服务都有一个与之关联的 Envoy sidecar 代理，它负责执行流量管理、负载均衡、故障恢复、安全性等功能。

**Pilot:** 它负责服务发现和动态配置 Envoy 代理。它监控运行中的服务实例，并将配置信息下发到 Envoy 代理，使得流量路由和负载均衡等功能能够动态适应服务拓扑的变化。它能够提供弹性和高可用性，确保 Envoy 代理能够及时获得最新的服务信息。

**Citadel:** 它是 Istio 的安全组件，负责实现服务间的安全通信。它处理服务身份认证、凭证分发和流量加密等功能，确保服务之间的通信是安全的。它提供了强大的身份和凭证管理，支持服务间的强制性加密。

**Galley:** 负责验证和转换 Istio 配置，确保其符合 Envoy 的规范。它充当配置的中心点，协调各个组件之间的配置信息。

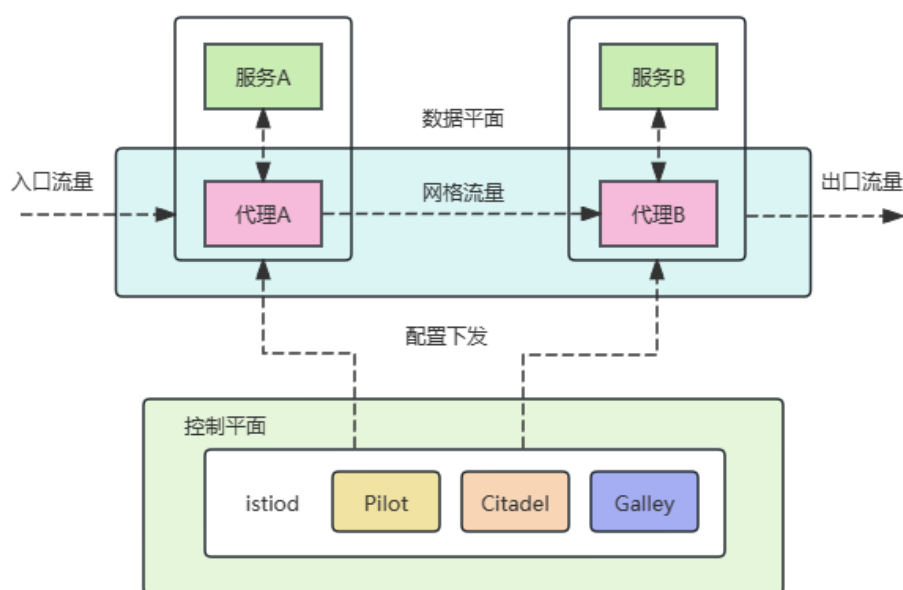


图 2.4 Istio 架构图

## 2.2 服务治理技术

服务治理技术可以细分为服务注册与发现、负载均衡、服务熔断限流等几个方向。服务治理本身就是一个在微服务架构之后才产生的一个概念，从最早的单体系统架构，这个时候所有系统服务都在一个包或者服务当中，所以不存在服务治理的问题，随着系统架构的演进和发展，架构逐渐从单体向微服务架构转变，系统中的服务被拆分成多个业务模块，分别开发和部署，这就涉及到了多个微服务模块之间如何进行远程服务通信，如何保证远程通信的可靠稳定运行，远程流量的负载问题以及当远程服务不可用时保证系统可用性的方案<sup>[39]</sup>。

在微服务架构时代，主要是通过中间件的形式来完成微服务的治理，通过将中间件的 SDK 集成到各个微服务的业务模块包当中，并且在代码中编写相应的服务治理逻辑来实现服务治理<sup>[40]</sup>。但是到了服务网格时代，网格会将系统中的业务代码和服务治理代码进行解耦，直接以非入侵的方式来进行服务治理，通常以 Sidecar 边车模式的方式来进行网络代理，在这种情况下，服务的所有流量都会交给 Sidecar 代理来进行处理，服务网格通过配置下发的方式来控制每个微服务对应代理的服务治理行为，通过这种方式，以 Istio 为代表的服务网格技术就可以统一管理整个网格中的数据流量，实现脱离服务的治理<sup>[41]</sup>。

### 2.2.1 侵入式的服务治理技术

侵入式的服务治理技术主要是通过将服务治理相关的中间件 SDK 集成到各个微服务模块当中，在微服务模块中编写对应的服务治理代码，并且启动对应的中间件客户端，以这种方式实现微服务治理<sup>[42]</sup>。在重构前的电力营销系统当中，服务注册发现、服务间通信、服务熔断限流降级都是通过 Spring Cloud Alibaba 的中间件来实现的。所以本章侵入式的服务治理介绍就以 Spring Cloud Alibaba 为例。微服务架构下的服务注册和发现是通过 Nacos 来实现的，图 2.5 是 Nacos 的工作流程图。

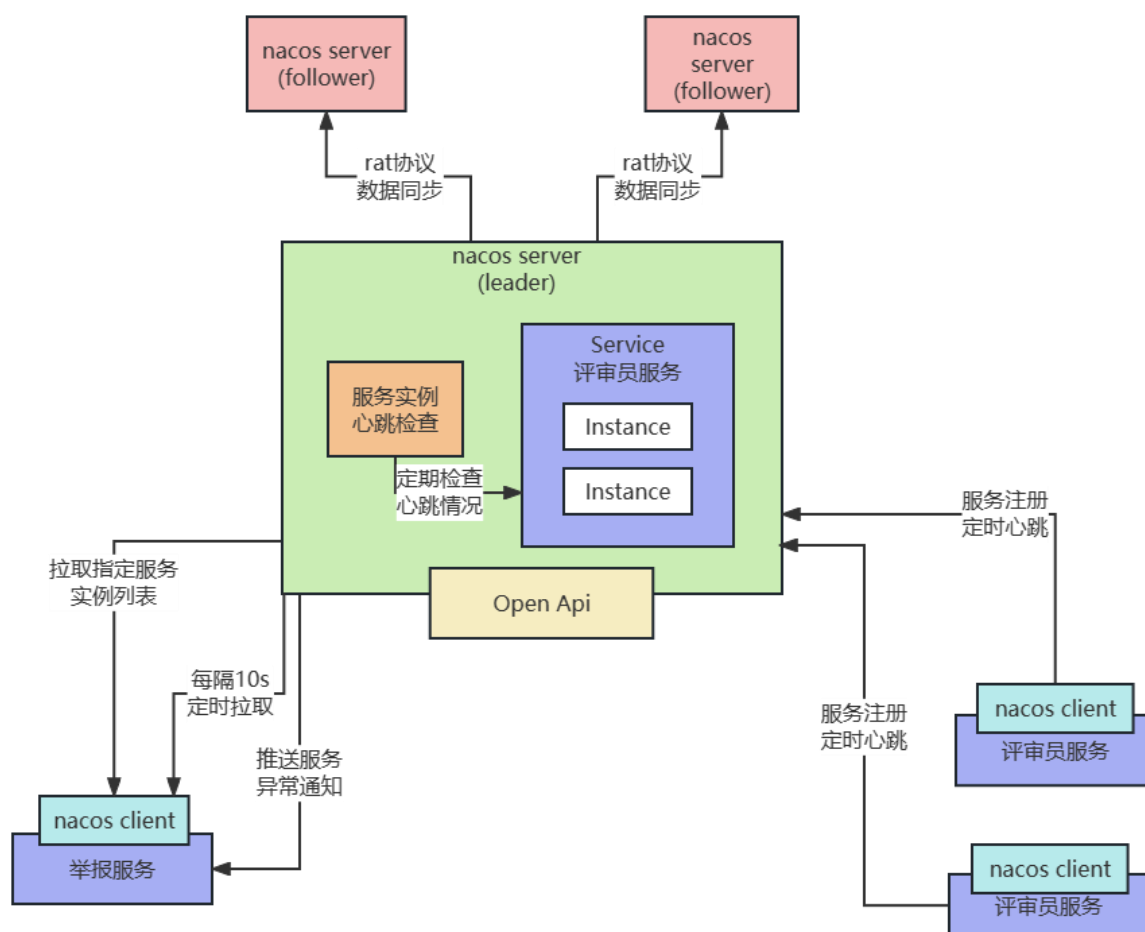


图 2.5 Nacos 工作流程

首先启动 Nacos 服务端，通过在微服务模块当中引入 Nacos，在代码中配置 Nacos 服务端信息即可将微服务注册到 Nacos 注册中心当中，服务消费者通过 Nacos 的服务发现功能获取服务实例的信息。服务消费者在获取服务实例列表后，通过客户端负载均衡算法选择一个实例来发送请求。常见的负载均衡算法包括轮询、随机、加权轮询、加权随机等<sup>[43]</sup>。Nacos 通过定期的心跳检测来监测服务实例的健康状况。服务提供者定期向 Nacos 发送心跳，Nacos 根据心跳的响应情况来判断服务实例是否健康。如果



Nacos 检测到某个服务实例不再健康，它将从服务注册表中移除该实例。消费者在获取服务实例列表时，将不再收到不健康的实例，从而实现故障转移<sup>[44]</sup>。

微服务架构下的远程调用是通过 OpenFeign 来实现的。OpenFeign 通过结合 Spring Cloud 和 Ribbon 等组件，实现了简单且优雅的服务间通信。开发者使用 Java 接口定义远程服务的 API。在应用启动时，Spring 容器会扫描这些接口，并生成相应的代理类。这个代理类实现了接口定义中的方法，并包含服务调用的逻辑。OpenFeign 集成了 Ribbon，通过服务名实现负载均衡<sup>[45]</sup>。Ribbon 会在运行时实现请求的负载均衡。生成的代理类通过动态代理技术，将接口方法的调用转发到 Feign 的运行时组件。在这个过程中，Feign 会将映射后的 HTTP 请求发送到远程服务。获取到远程服务的响应后，Feign 将响应的内容解析为接口方法期望的返回类型。

微服务架构下的熔断器实现主要是依靠阿里巴巴开源的 Sentinel 中间件，它主要是负责分布式系统当中的流量控制以及为了保证服务高可用而设计的熔断和限流策略。Sentinel 允许开发者定义一些需要监控的资源对象，Sentinel 客户端会监控这些资源对象上的流量并且根据开发者定义的规则对这些资源对象进行限制。“熔断器”就向他的名字一样，在整个微服务系统当中，是起着保护整个系统服务稳定的作用。当某个下游服务出现故障时，上游调用下游服务失败就会有对应的降级策略，来保证上游服务可用，而不是被下游服务带崩，在一些流量大的场景下，还需要对服务进行限流，避免过大的流量打入服务器导致服务器无法承受压力而崩溃。

### 2.2.2 非侵入式的服务治理技术

最具代表性的非入侵的服务治理技术体系就是 Service Mesh，例如 Istio，它将自身的系统架构分为控制平面和数据平面本质就是因为其核心思想是非侵入式的服务治理技术，服务治理是 Istio 的核心功能，它包括服务注册发现、流量的负载均衡和分发、限流和熔断、系统监控与可观察性等机制<sup>[46]</sup>。它能够在实现微服务架构下服务治理需求的同时，减少中间件的依赖引入，避免分布式项目中间件依赖版本碎片化的问题，下面将对这些方面进行详细介绍。

服务注册与发现是分布式系统中的基本组成部分，它允许服务动态地加入或离开系统，并确保其他服务能够找到它们<sup>[47]</sup>。如图 2.6 所示是 Istio 的服务注册与发现的工作流程，在 Istio 中，这一功能基于 Istio 的服务注册表实现。服务在部署时通过 Istio 的 Sidecar 代理将自己注册到服务注册表中，使其他服务能够发现它们。当服务正常启动，该服务因为已经在注册表中注册过，就可以被其他网格中的服务发现<sup>[48]</sup>。服务发现是



Envoy 代理使用负载均衡算法，例如轮询、随机或基于权重的算法，以确保流量在可用的服务实例之间均匀分布<sup>[50]</sup>。如图 2.7 所示，Istio 还允许在流量分发时进行智能路由，Istio 能够根据请求的特定标签或头信息将流量引导到不同的服务版本或环境中。这使得在不影响整体系统的情况下进行金丝雀发布或 A/B 测试成为可能<sup>[51]</sup>。同时如果金丝雀发布过程中出现问题，服务网格可以快速回滚到稳定版本，而无需手动介入。这可以大大减少服务中断时间，提高系统的可靠性和可用性。

限流和熔断是在微服务架构中确保系统稳定性的关键机制。Istio 提供了对这些功能的直接支持，如图 2.8 所示。Istio 允许在服务之间设置流量限制，以确保某个服务不会因为过多的请求而过载。通过配置 Envoy 代理的限流规则，可以在不同的服务间实现细粒度的流量控制<sup>[52]</sup>。熔断是一种防止服务连锁故障的机制。Envoy 代理通过熔断器实现熔断功能，当某个服务出现故障或延迟时，熔断器会暂时停止对该服务的请求，防止故障扩散到整个系统。Istio 通过其强大的服务网格架构，为微服务应用提供了限流和熔断等功能，能够避免引入服务治理中间件，降低代码耦合度，还能够帮助开发人员轻松管理微服务，提高系统的可维护性、可扩展性和可用性。

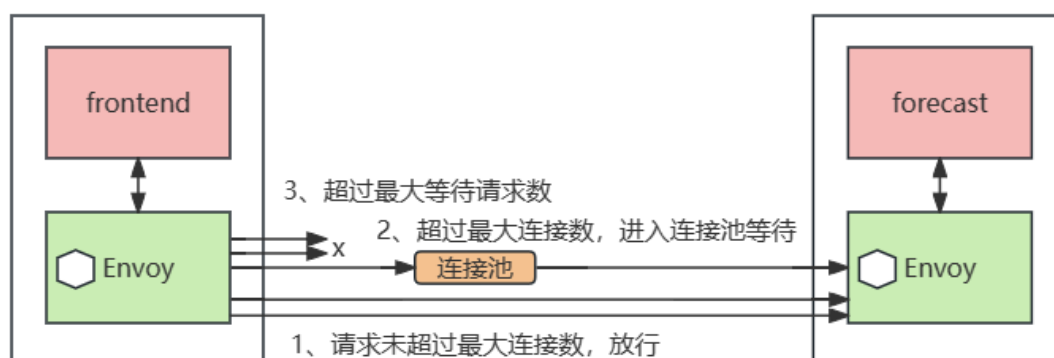


图 2.8 Istio 限流降级过程

## 2.3 本章小结

本章首先对容器化技术 Docker、容器编排技术 Kubernetes、服务网格技术 Istio 的原理和架构进行了详细的介绍，阐明了本文选择这些技术的原因。接下来针对传统微服务架构场景下的侵入式服务治理方式以及云原生服务网格架构下的非侵入式服务治理方式进行了分析，阐明了非侵入式服务治理方式的优势，为后文将现有的基于微服务架构的电力营销系统重构为云原生架构以及对 Kubernetes 资源调度算法的优化奠定基础。

## 第3章 云原生架构服务治理方案设计与应用研究

为了解决所在实习企业的“电力营销系统”项目服务治理代码和业务代码耦合度高和中间件版本碎片化问题。本章将对整个系统进行重构，将系统从依托于侵入式 SDK 的微服务架构重构为非侵入式的、依托于服务网格的云原生架构。为了更加适配 Istio，本章将设计并实现一个轻量化的服务通信组件。为了解决 Kubernetes 默认资源调度器在本系统运行过程中资源调度不均衡的问题，提出了基于 AHP-熵权法的多指标资源调度算法 MMRSA，使资源调度器能够感知到节点网络带宽以及磁盘 I/O 利用率并依据多个维度的数据进行打分，同时通过算法进行更加合理的权重分配，以实现更均衡的资源调度。

### 3.1 服务治理方案设计与分析

本文从网关与集群流量下发治理方案重构与集群内服务治理方案重构两个角度来进行系统重构方案设计。集群流量下发治理方案重构部分将包含服务注册和发现的重构，因为这两者密不可分。集群内服务治理方案重构部分主要是进行服务熔断、限流、降级部分的重构分析与设计。

#### 3.1.1 网关与集群流量下发治理方案

首先，这里需要分析一下原电力营销技术支持系统的网关实现，网关主要是用于处理整个系统当中的南北流量，旧的网关是依靠集成 Spring Cloud 的 Spring Cloud Gateway 来实现的。如图 3.1 所示，Spring Cloud Gateway 网关主要是作为整个系统后台服务的流量入口，外部请求后台服务的所有南北向的接口流量都需要流经网关，同时负责将所有外部请求流量根据条件进行流量分发，将请求分发给对应的微服务进行处理。在以 Spring Cloud 为核心的服务治理体系当中，网关是需要以一个单独的微服务模块来部署的，网关不同于其他业务相关的微服务模块，在进行模块开发的时候也具有一定的特殊性。使用 Spring Cloud Gateway 来实现网关已经大大降低了开发人员开发的工作量，同时他也提供了一些可扩展的服务治理功能，但是其入侵式的开发方式，随着项目体量的扩大，会给项目带来一些问题，在需要对网关路由策略做出调整的时候，就不得不去修改 Spring Cloud Gateway 的后台业务代码，同时进行停机更换、重新

部署模块，这就给项目的稳定运行增加了隐患。所以本文采用非入侵式的网关来替代 Spring Cloud Gateway 成为新电力营销系统架构的网关技术解决方案。

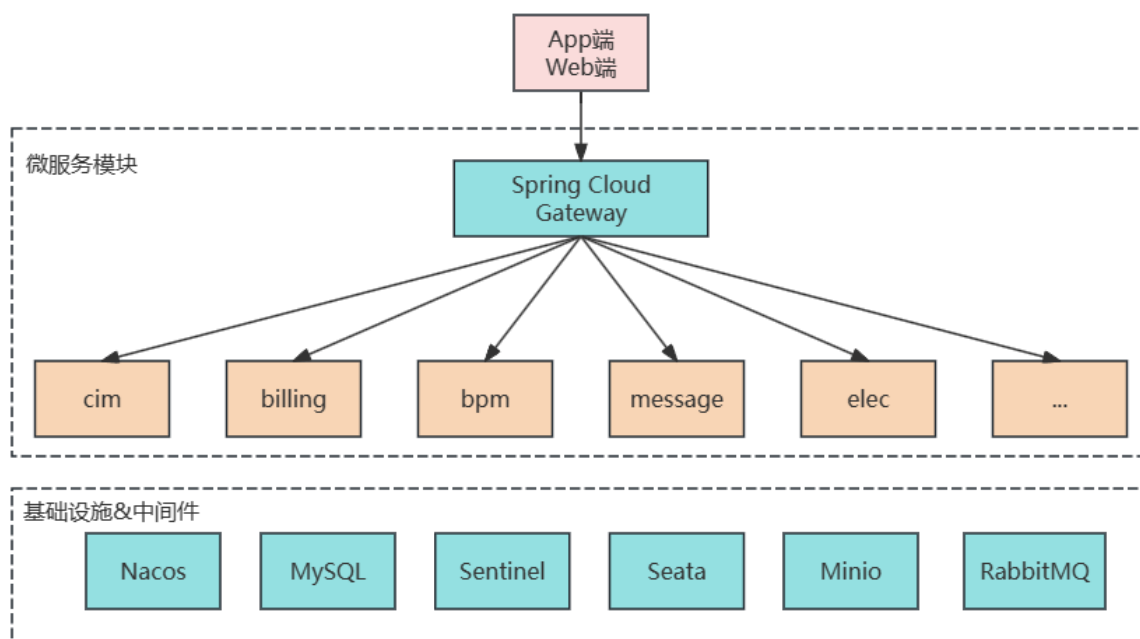


图 3.1 原电力营销系统网关架构图

实际上，Kubernetes 在不引入中间件的情况下就可以支持这样的网关技术，它自身提供一种叫 Ingress 的 CRD，通过 Ingress，可以实现 Spring Cloud Gateway 作为流量入口以及流量分发的作用。通过使用 Kubernetes 所提供的 Ingress，可以在不引入额外网关组件的情况下实现对入口流量的分发任务。然而，随着 Service Mesh 技术的发展与演进，以 Kubernetes 为基础的 Istio 拥有一个更为强大的网关 Istio Ingress Gateway，它与 Kubernetes 的 Ingress 相比，支持更为高级的路由规则，例如基于请求头，查询参数的匹配，还能够实现更加灵活的定义流量的路由规则，灵活性更高。同时因为本文打算将系统重构为 Service Mesh 架构，Istio Ingress Gateway 作为 Istio 的一部分，与 Istio 的其他部分的集成度也更高，同时因为 Istio Ingress Gateway 更加灵活的流量路由规则，使得开发人员在整个软件开发迭代流程中，可以更加轻易的实现服务版本的灰度发布以及金丝雀发布。综上所述，本文最终选择 Istio Ingress Gateway 作为重构后的网关解决方案。

如图 3.2 是 Istio Ingress Gateway 的流量下发图，外部的流量发送到 Istio Ingress Gateway 之后，Istio Ingress Gateway 会根据 Istio 定义的 Virtual Service 中的流量分发策略将流量分发到对应的 Service 当中，然后 Service 会将所有流量分配给属于自己的 pod，

pod 中包含的就是每一个微服务模块和它所对应的 sidercar 代理，sidercar 在接收到 Service 分发过来的流量后将流量转发给微服务所在 pod，这就是 Istio 南北向流量路由的过程。

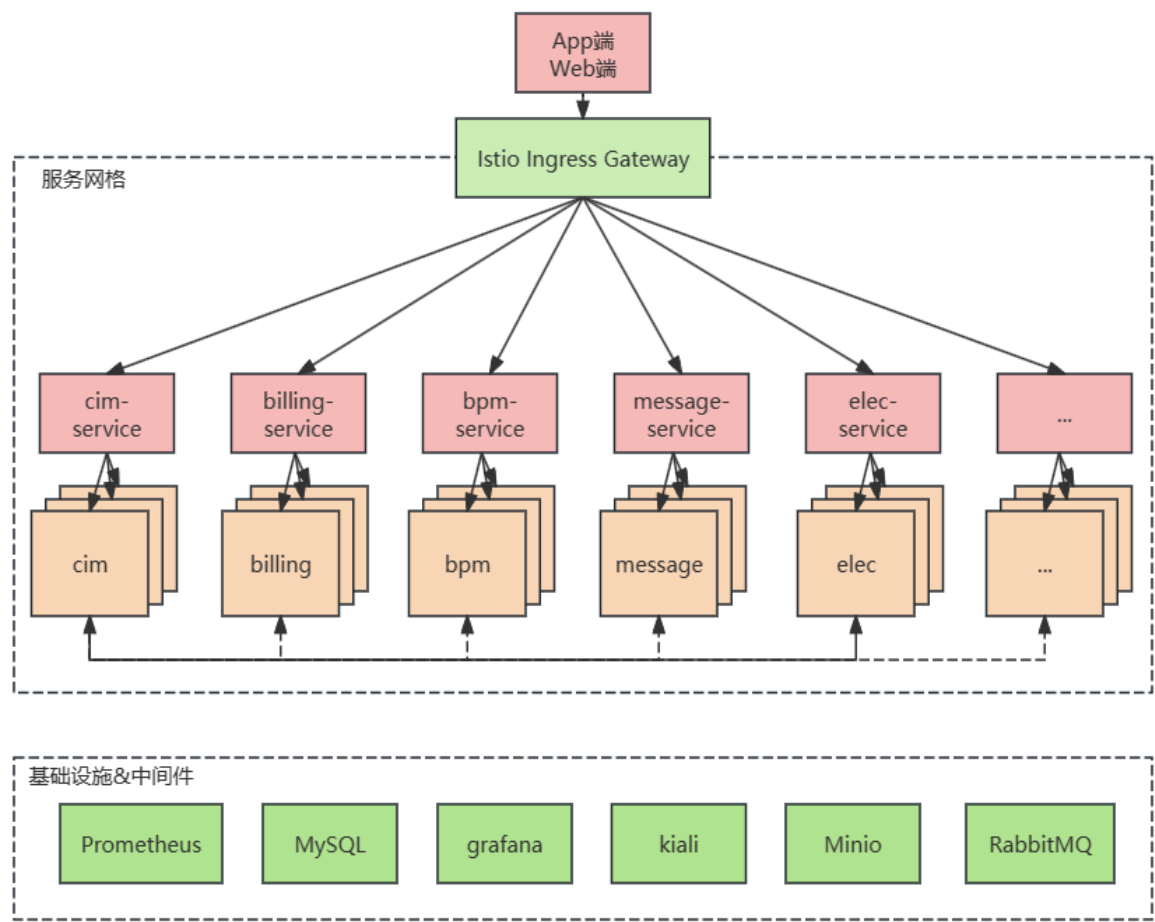


图 3.2 云原生架构下的网关架构图

上面介绍的网关的重构可以理解为对整个系统当中南北向流量的重构，接下来对系统内部的流量路由进行重构，也就是重新规划系统内的东西向流量，在介绍重构方案之前，需要先对系统服务注册和发现机制进行重构。

在原有的电力营销系统当中，服务注册与发现是通过 Nacos 来实现的，如图 3.3 是 Nacos 流量分发流程图。当一个微服务启动时，它会向 Nacos 注册自己的信息，包括服务名、IP 地址、端口等。这样，Nacos 就会记录这个服务的存在，并成为服务的注册中心。服务的注册信息包含在 Nacos 的服务管理中。当一个微服务需要调用其他服务时，它可以向 Nacos 查询需要调用的服务的信息。Nacos 会返回一个或多个可用的服务实例，包括它们的网络地址等信息。服务消费者可以根据这些信息直接与提供服务的实例进行通信。

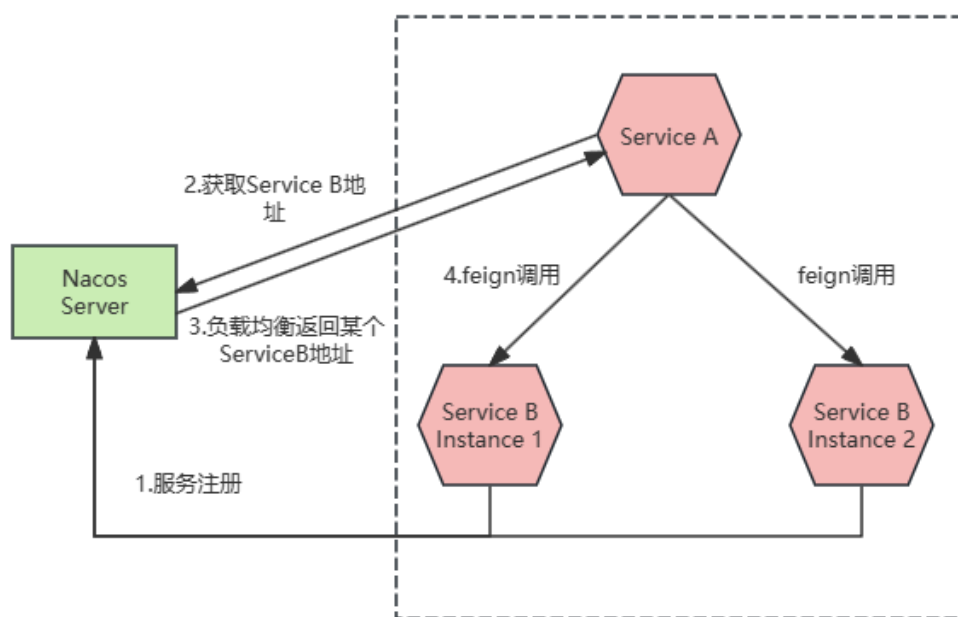


图 3.3 原架构下的 Nacos 流量分发流程图

本次重构将原有的依赖于 Nacos 的服务发现和服务注册功能交给 Kubernetes 的服务注册发现功能来实现。对于每一个 Node，都运行着一个 DNS 插件，负责处理 DNS 查询请求。运行在每个 Node 上的 Kubernetes 代理 Kubelet 会监控集群中的 Service 和 Endpoint 对象的变化，并将这些信息报告给 DNS 插件。每当集群中有新的 Service 或者 Endpoint 创建的时候，都会实时的反馈给 DNS 插件，然后更新 DNS 记录。DNS 记录保存的就是 Service 在 Kubernetes 集群内的域名以及与他相关联的 Pod 的 IP 地址信息，Kubernetes 的服务注册流程如图 3.4 所示。

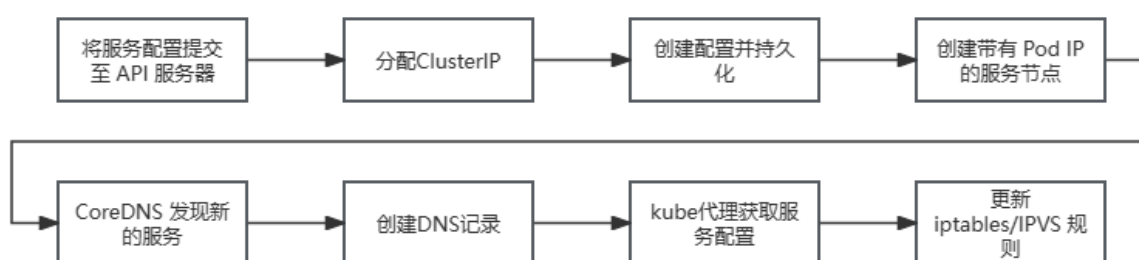


图 3.4 Kubernetes 服务注册流程图

如图 3.5 是 Kubernetes 的服务发现流程图。当某个 pod 想要远程调用的时候，这个 Pod 需要用 Service 连接其它 Pod。首先向集群 DNS 发出查询，把 Service 名称解析为 ClusterIP，然后把流量发送给位于 Service 网络的 ClusterIP 上。Pod 把流量发送给它的缺省网关。这一行为导致流量被转发给 Pod 所在节点的网卡，然后是节点的缺省网关。这个操作中，节点的内核修改了数据包 Header 中的目标 IP，使其转向健康的 Pod。



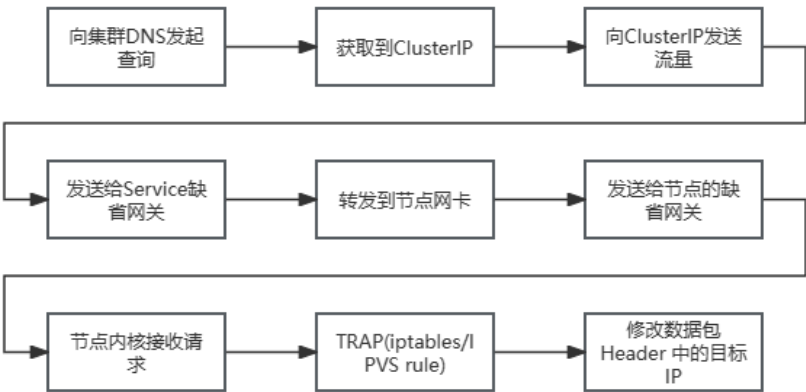


图 3.5 Kubernetes 服务发现流程图

以上就是对系统服务注册和发现机制的重构方案，现在进行系统内部东西向流量的重新规划，旧的系统架构下，需要依靠 Nacos 来完成东西向流量的路由。后台的各个微服务在服务启动的时候会将自己的地址信息与服务信息绑定注册到 Nacos 上，当其他服务要远程访问的时候，会去 Nacos 上面获取到当前处于可用状态的远程服务的地址信息，然后再将请求发送到该地址上的微服务当中，完成远程通信。如果某个服务有多个副本，Nacos 也会根据具体的配置来完成如轮询、随机等方式的负载均衡。

在新的系统架构下，整个系统东西向的流量分发和负载均衡全部都交给服务网格来完成，如图 3.6 所示。服务发现是通过 Kubernetes 来实现的，当服务 A 想要去访问服务 B 的时候，服务 A 会通过 Kubernetes 找到服务 B 的 Service 对应的集群内 ip 地址，然后根据该地址进行请求发送，接收到服务 A 发送过来请求的 Service B 会将请求通过负载均衡的方式分发给其对应的可用的 Pod B，实现网格中东西向流量的路由。

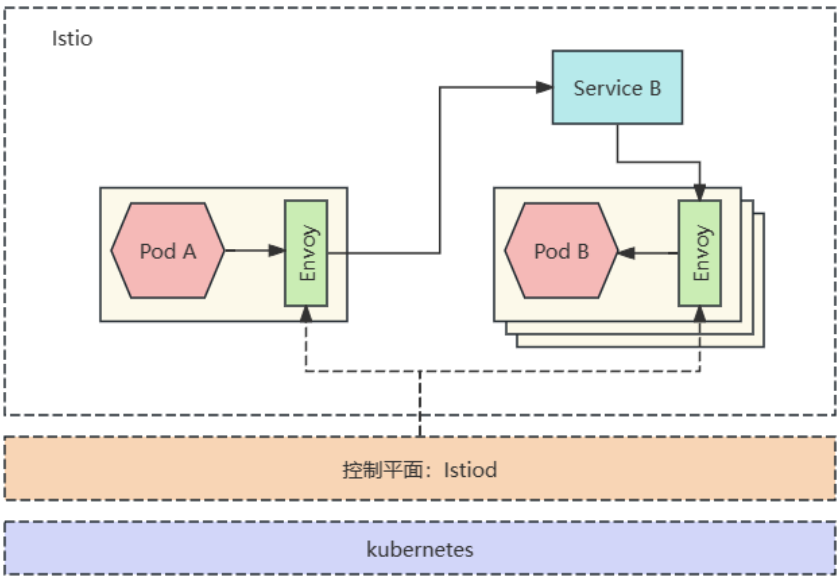


图 3.6 Istio 东西向流量分发流程图



### 3.1.2 集群内服务治理方案

由于上一节在进行流量路由重构的时候，已经对服务发现和注册的实现进行了重构设计，所以本节将只介绍熔断、限流、降级的服务网格重构方案。

在系统重构前，熔断限流和降级是依托于阿里巴巴开源的 Sentinel 来实现的，在 Sentinel 里面，所有的资源都对应一个资产名称以及一个 Entry。Entry 可以通过对主流框架的适配自动创建，也可以通过注解的方式或调用 API 显示创建；每一个 Entry 创建的时候，同时也会创建一系列功能插槽（slot chain），这些插槽有不同的职责，通过责任链方式进行构建，整体框架如图 3.7 所示。

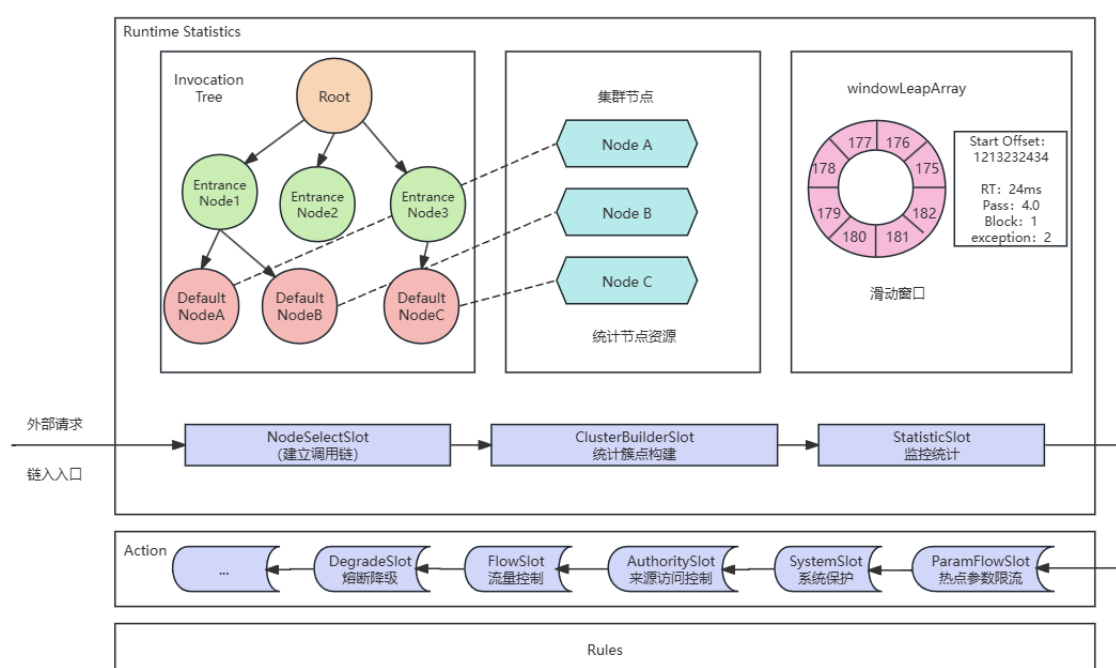


图 3.7 Sentinel 核心架构图

用 Sentinel 虽然可以实现更细粒度的熔断限流降级，但是也同样限制了系统中微服务的开发语言，Sentinel 高度依赖 java 语言，无法管理其他语言开发的服务。为了让系统具有更高的可扩展性，本文通过 Istio 取代 Sentinel 来实现熔断限流与降级。由于本系统中的限流场景没有那么复杂，所以本文对于服务限流的重构采用的方案是本地限流的方式。同时由于限流方式改为了非入侵的方式，所以限流的方式也从服务的内部迁移到了服务外部，就不能够像 Sentinel 一样实现代码块级别粒度下的限流操作，只能根据 HTTP 请求的 URL 进行接口粒度的限流。

重构后的服务熔断与降级依赖于 Istio 的 Envoy 代理，如图 3.8 所示。首先通过发布 yaml 声明式配置文件的方式，将配置更新到服务网格控制平面当中，然后控制平面

再将配置进行下发，下发到每一个数据平面的 Envoy 当中，Envoy 代理内置了熔断器和健康检查的功能，用于根据配置的规则自动执行熔断和降级操作。熔断可以通过配置 Istio 的 DestinationRule 对象，可以定义熔断的阈值、超时时间等参数。降级可以使用 VirtualService 和 DestinationRule 来配置请求失败时的备用路由，这样在触发降级策略的时候，就知道将流量路由到哪里。

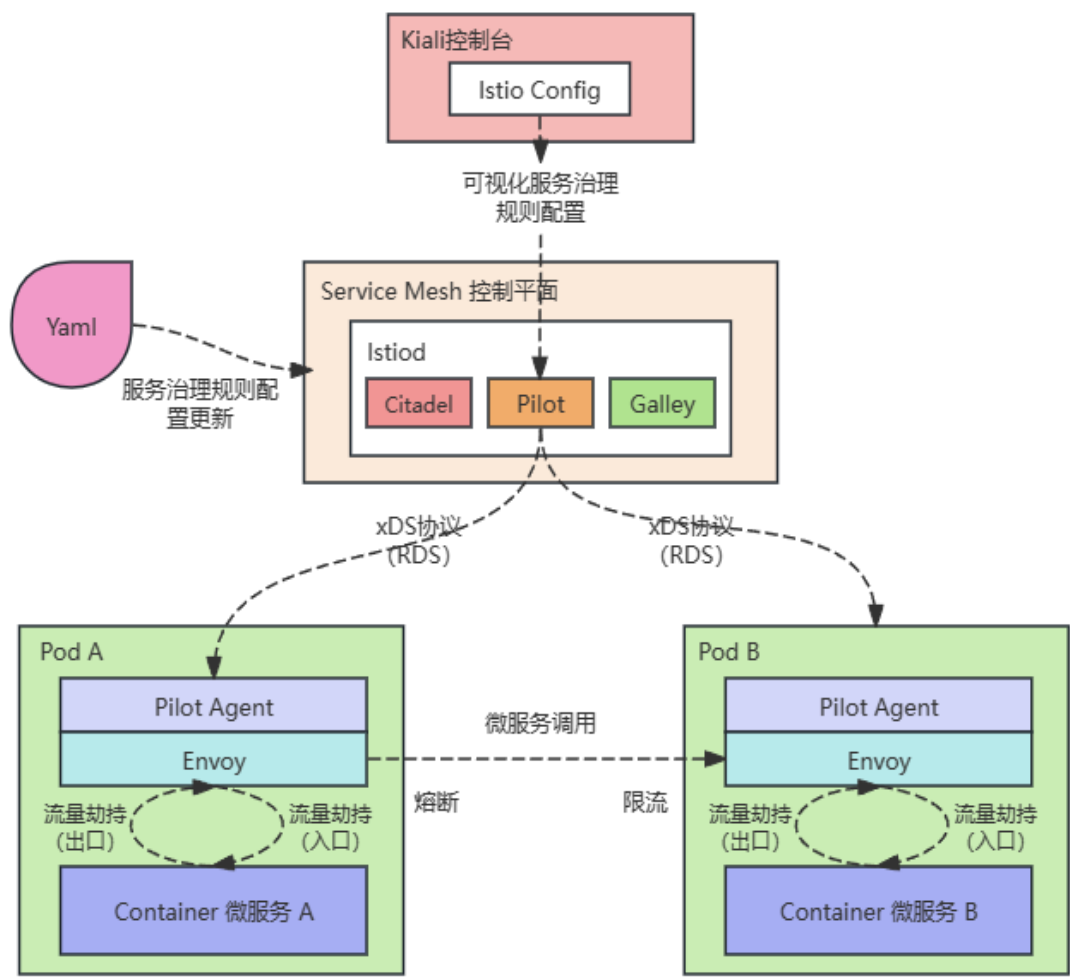


图 3.8 Istio 熔断限流原理图

### 3.2 服务通信组件的需求分析与设计

由于本文对原电力营销系统的服务治理框架进行了重构，重构后的系统是基于服务网格的服务治理方式，而服务网格自身就可以提供负载均衡、熔断限流能力，开源的组件和本系统架构不够适配，所以本文设计实现了一款轻量化、响应快的服务通信组件以适配基于服务网格的系统架构。

### 3.2.1 服务通信组件的需求分析

原电力营销系统当中，整个项目的架构采用的都是围绕 Spring Cloud 为核心的微服务架构下的治理组件，服务的治理逻辑和服务的业务逻辑是紧密耦合的。远程通信使用的是 Spring Cloud OpenFeign 组件来完成的，它主要是通过声明式的方式来定义要实现远程服务调用的接口，当服务启动，这些提前定义好的接口会生成对应的 RPC 代理类，远程通信就依赖这个被封装了的代理类来完成。OpenFeign 还集成了 Ribbon，通过 Ribbon 实现 Openfeign 的负载均衡功能，Ribbon 还可以从服务注册中心获取到其他远程服务的地址。同时 Openfeign 还可以集成服务注册中心以及熔断器等来配合注册中心以及熔断器实现远程通信以及熔断。

OpenFeign 是非常优秀的通信组件，它是在市面上应用最多的服务间通信组件，它通过声明式的方式大大简化了远程调用的开发复杂度，同时屏蔽了很多实现的具体细节，方便开发者快速开发上手，而且与其他 Spring Cloud 组件良好集成，易扩展。但是因为本文重构后的系统底层基于服务网格，服务网格本身就支持网格中模块之间通信的负载均衡以及熔断限流等流量监控和流量控制的操作。所以对于基于服务网格的微服务模块，现有的基于微服务架构的开源服务通信组件都不够适配基于服务网格架构的系统，所以本文设计了一个适配服务网格架构的更加轻量级的服务通信组件，图 3.9 是服务通信组件的简略时序图。

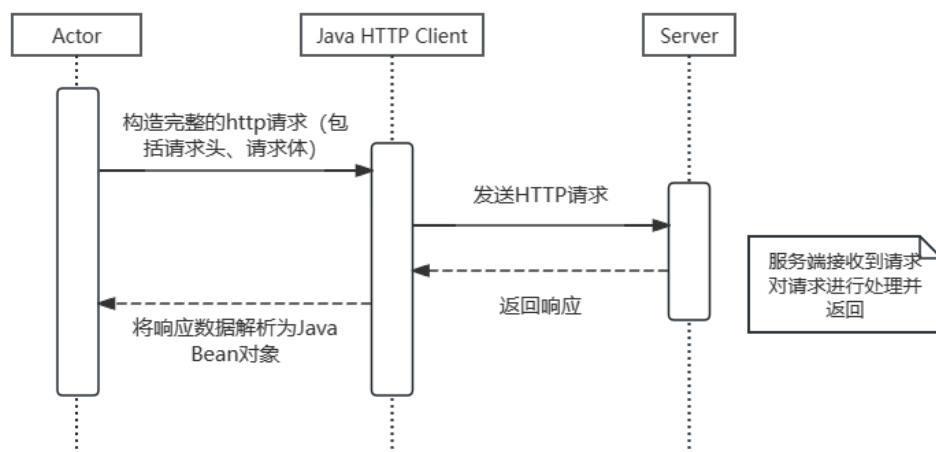


图 3.9 服务通信组件关键流程时序图

为了使该通信组件能够适配服务网格架构的通信需求，在更加轻量化的同时依照开发者常用的通信组件的开发习惯进行开发、降低重构代码时的工作量并且方便开发者更快上手使用，本文确定该通信组件的需求如下：

- (1) 通信组件需要支持多环境配置，不光要支持集群中的服务通信，也应该支持

## 本地开发调试的方式

因为该通信组件的应用场景主要是在服务网格内部，但是开发者在开发过程中，难免会涉及到本地启动项目去调试代码，进行功能测试等非业务需求，同时，开发者的电脑往往并不会在服务网格的集群内部，如果不对线上环境外的开发环境进行适配的话，开发者将难以应用该通信组件完成编码任务的本地测试。所以本文通过读取代码中指定配置文件指定字段的方式来进行多种环境的适配。达到支持如集群内部、集群外部等多种环境下的开发任务需求。

### （2）将远程通信的具体细节封装起来，要适配服务网格架构中的远程通信

对于重构后的基于服务网格进行治理的项目，虽然开源的服务通信组件不够轻量化，其中包含了很多冗余的功能，并不够适配，但是也包含着一些比较优秀的实现思路，本通信组件也需要将具体的远程调用细节封装起来，让开发者使用本组件的时候就像调用自己的本地服务一样，这就需要将包括扫描接口、创建代理、构造请求、编解码、加解密、发送接收请求等过程封装到本组件当中，让服务调用者只需关心远程调用本身，而不需要关心具体的底层实现细节。

### （3）要支持服务监控中的链路追踪功能，适配 jaeger、zipkin 链路追踪

在服务网格当中，服务的治理至关重要，同样的，对于网格中服务的可观测性也一样重要，例如网格中流量的链路追踪，需要借助 zipkin，本组件也需要支持链路追踪，保证可以根据检测系统中查到的问题 id 进行链路追踪排查错误，保证系统稳定性。因为服务通信组件是基于 HTTP 协议来进行通信的，为了支持 zipkin 等链路追踪功能，各个微服务在通信的时候就必须携带上一些具有标识性的能够确定具体链路的 head 信息，所以本服务通信组件就需要对一些关键的 HTTP head 进行透传，包括“x-request-id”，“x-b3-traceid”，“x-b3-spanid”，“x-b3-sampled”，“x-b3-parentspanid”，“x-ot-span-context”，“x-datadog-trace-id”，“x-datadog-parent-id”，“x-datadog-sampled”。这些字段的意义如下：

“x-request-id”用于唯一标识一个请求，对于链路追踪而言，这个标识可以用于关联同一个请求在不同服务之间的调用。

“x-b3-traceid”，“x-b3-spanid”，“x-b3-sampled”，“x-b3-parentspanid”通常用于 B3 规范（Brave Distributed Tracing），用于在微服务架构中进行分布式跟踪。其中：

“x-b3-traceid”表示跟踪整个请求的唯一标识。

“x-b3-spanid 表示某个服务实例内部的一个操作的唯一标识。

“x-b3-sampled 表示是否对这次请求进行采样，以确定是否记录跟踪信息。

“x-b3-parentspanid 表示当前操作的上级操作的唯一标识。

“x-ot-span-context”用于传递跨服务的上下文信息。类似于 B3 头部，帮助实现跨服务的分布式跟踪。

“x-datadog-trace-id”，“x-datadog-parent-id”，“x-datadog-sampled”通常用于 Datadog APM，用于实现跟踪和性能监控。其中：

“x-datadog-trace-id 表示跟踪整个请求的唯一标识。

“x-datadog-parent-id 表示当前操作的上级操作的唯一标识。

“x-datadog-sampled”表示是否对这次请求进行采样。

#### （4）保证通信组件的易用性

因为原系统中的采用的是开源的 Spring Cloud OpenFeign 作为服务通信组件，OpenFeign 在开发使用过程中是一个声明式的组件，用户通过声明接口的方式来实现远程调用接口的定义，为了符合现有的编程习惯和常规的开发逻辑，本通信组件依旧会采用声明式的方式，在降低学习成本，不改变编码风格的同时完成组件开发。

#### （5）降低耦合度，保证通信组件轻量化

为了保证组件足够的轻量化，在开发过程中需要避免引入太多的第三方依赖库，这样可以使本组件在保证通信效果稳定的情况下相较于开源的服务通信组件足够的小巧。

### 3.2.2 服务通信组件的总体设计

首先本服务通信组件为了降低重构系统的成本，在设计阶段着重考虑了旧有组件的编码习惯和开发逻辑，通过自定义注解来完成声明式的配置。如图 3.10 是服务通信组件生成接口代理的时序图，在服务启动的时候，Spring 框架会查找所有使用 `@EnableIstioClients` 注解的配置类。这个注解通过 `@Import(IstioClientsRegistrar.class)` 引入 `IstioClientsRegistrar`，这是一个重要的组件，负责后续的扫描和注册过程。`IstioClientsRegistrar` 是一个实现了 `ImportBeanDefinitionRegistrar` 接口的类。Spring 在处理 `@Import` 注解时，会调用这个 `Registrar` 的 `registerBeanDefinitions` 方法。这个方法被 Spring 容器调用，用于注册由 `@IstioClient` 注解标记的接口。在该方法中，首先会注册默认的配置，然后调用 `registerIstioClients` 方法来处理 `@IstioClient` 注解。通过扫描自定义接口中的属性，筛选出需要生成对应代理类的 `@IstioClient` 接口。在 `IstioClientsRegistrar` 的 `registerFeignClients` 方法中，每一个被扫描到的接口都会执行 `registerFeignClient` 方法，这个方法负责创建和注册通信组件客户端的 Bean 定义，这里会创建一个 `IstioClientFactoryBean` 对象，这个对象实际负责代理对象的创建，它实现了

FactoryBean 接口，Spring 会调用它的 getObject 方法来创建代理对象。代理对象创建完成会被注入到 Spring 容器当中。当需要调用远程服务的时候，就会去 Spring 容器当中获取对应的对象来完成远程调用。

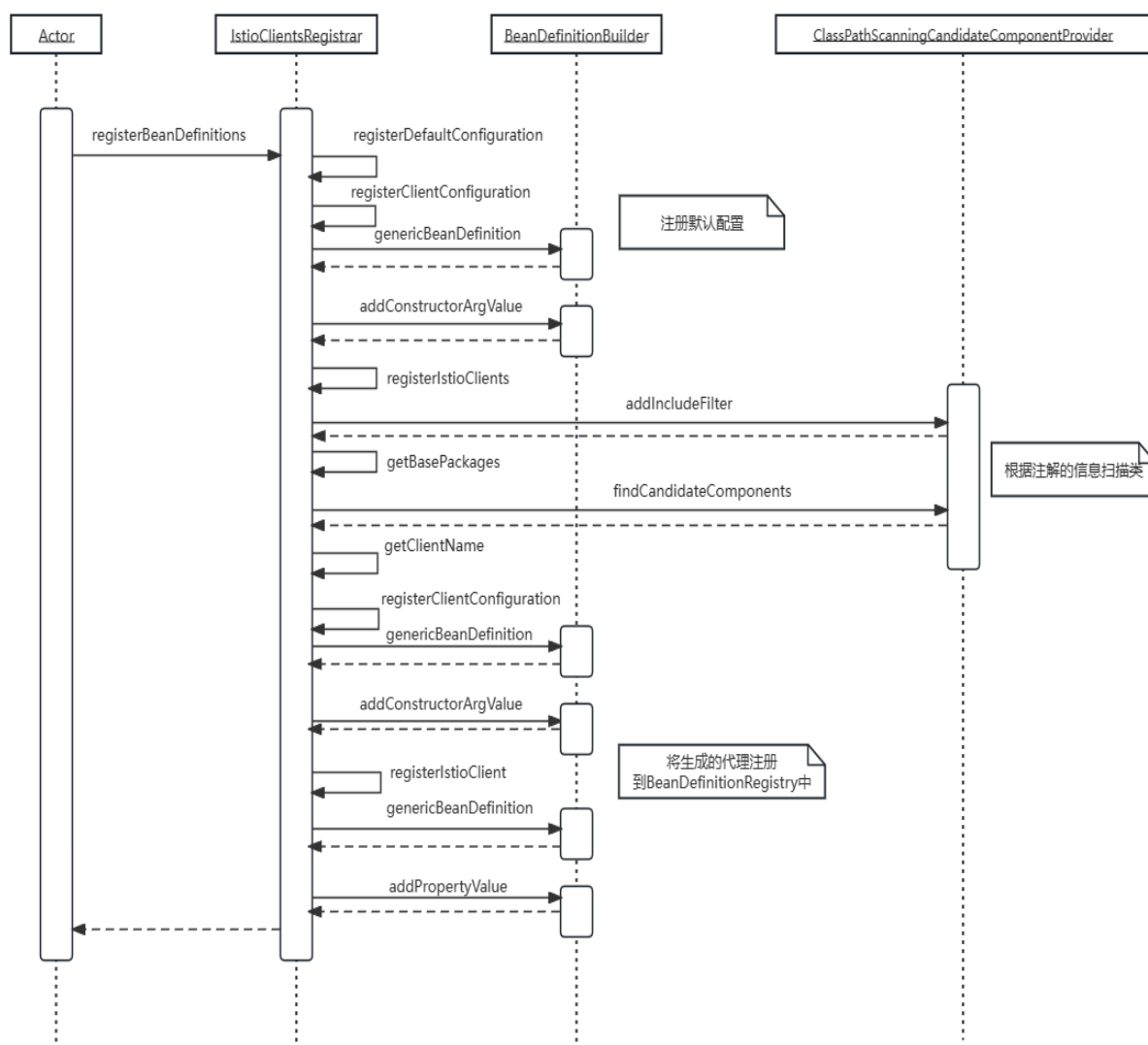


图 3.10 通信组件生成接口代理时序图

如图 3.11 是实际调用远程方法的时序图，本组件会先去容器中获取到对应的代理类，再通过代理类来实现远程方法的调用，当代理类的一个方法被调用时，Java 会自动调用 IstioInvocationHandler 中的 invoke 方法。IstioInvocationHandler 中的 invoke 方法会首先根据被调用的方法获取到对应的 MethodMetadata 对象，然后利用 MethodMetadata 构建一个请求模板 RequestTemplate，并将其传递给 SynchronousMethodHandler 的 invoke 方法。SynchronousMethodHandler 是本组件代理内部使用的处理器，它负责将方法调用转换为 HTTP 请求。

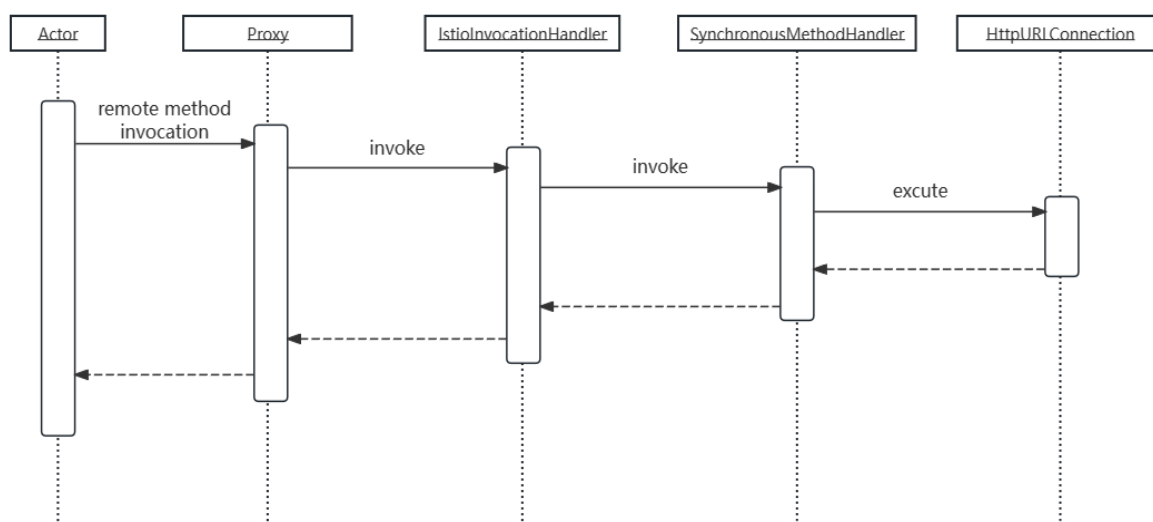


图 3.11 代理远程调用过程时序图

最后本组件通过集成的 OkHttp 组件向远程服务发送请求，在接收到远程服务的响应之后，本组件会使用解码器来将请求体中的信息解析成对象。

### 3.2.3 服务通信组件的详细设计

前面已经对本服务通信组件进行了逻辑流程上的总体设计，接下来将从更细粒度的角度对本组件进行设计，本文将服务通信组件在逻辑上分为 5 层来分别进行详细设计，每一层有着不同的职责。

#### (1) 接口层

这是系统的最顶层，定义了服务之间的通信接口，包括方法的名称、参数、返回值等信息。这一层描述了系统对外提供的功能，如图 3.12 是注解信息转化为内部属性的过程。在本服务通信组件当中，通过创建一些自定义的注解，来标注某些接口或包为特定的资源，例如：

**@EnableIstioClients:** 本接口用于通知 Spring Boot 应用程序要扫描哪些包以查找标记有 @IstioClient 注解的接口，并为它们创建代理类。这样就可以使用本组件来定义和调用远程服务接口。

**@IstioClient:** 注解可以定义一个接口，该接口描述了要调用的远程服务的各种方法。在接口中可以定义与远程服务对应的各种 HTTP 请求，包括请求方法、路径、请求体、请求头等信息。

**@GetMapping、@PostMapping:** 这些注解虽然不是本服务通信组件的注解，但是本组件在后续的封装过程中会用到这些注解来确定用户指定的具体 HTTP 请求类型。



在项目启动的时候，下一层解析层会对这些资源进行解析供代理层封装为代理对象。

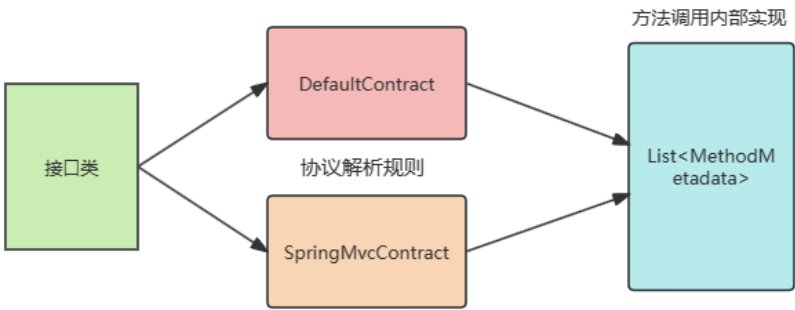


图 3.12 将接口类的注解信息解析为内部表现的过程

(2) 解析层

在这一层本组件会扫描标注了@EnableIstioClients 包下的@IstioClient 注解标记的接口，然后解析接口中的各个方法，包括方法的名称、返回值类型、方法参数等信息。对于每个方法，本组件会解析方法上的注解，包括 @GetMapping、@PostMapping、@PathVariable 等注解。这些注解提供了方法的 HTTP 请求信息，包括请求方法、路径、请求参数等。根据方法上的注解信息，本组件会构建对应的请求模板。请求模板包括了请求方法、路径、请求参数、请求体、请求头等信息，如图 3.13 所示。

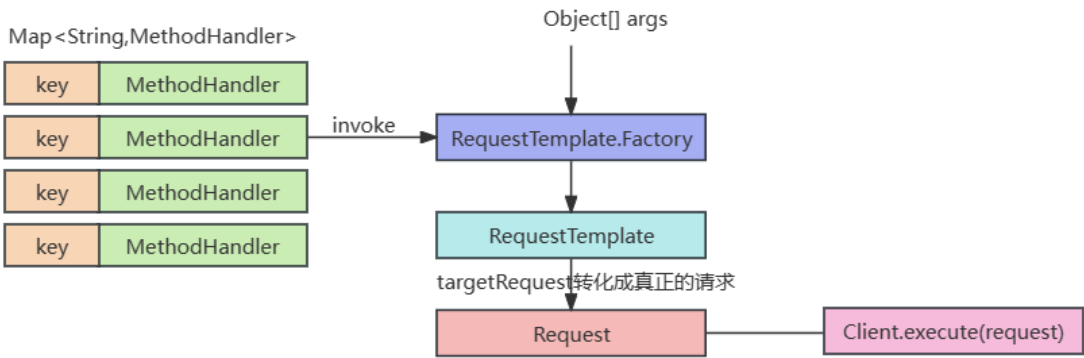


图 3.13 解析层动态生成 Request 的过程

(3) 代理层

这一层是在组件解析接口之后，会根据接口定义和注解信息来动态生成代理类，如图 3.14 所示。这个代理类会实现用户定义的接口，并覆盖其中的方法。生成的代理类会在运行时被加载到内存中，以便在调用时使用。在生成好代理对象之后会将其注入到容器当中，供后续实际远程调用的时候使用，这样用户就可以像调用本地服务一样调用远程服务，降低用户的感知度。



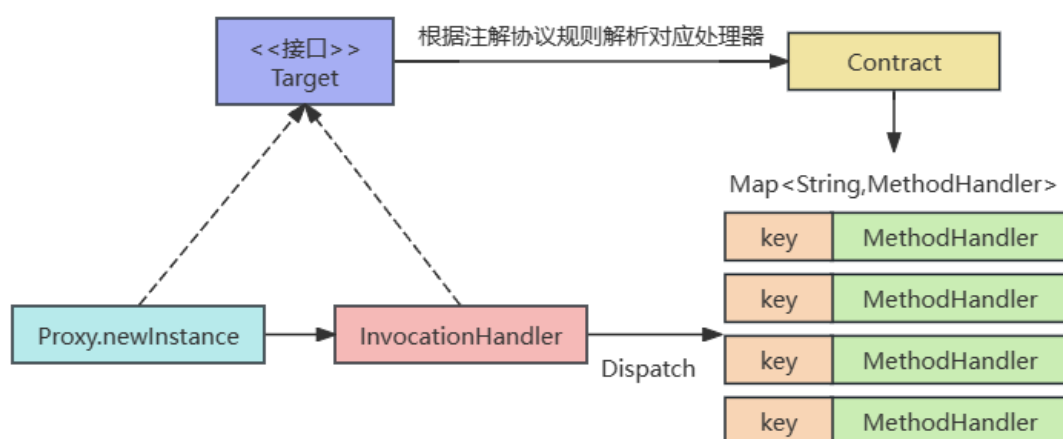


图 3.14 动态代理生成实现类的过程

#### (4) 编码层

在这一层，本组件负责将接口方法调用转化为 HTTP 请求，并将 HTTP 响应转化为方法调用的结果。这包括了请求参数的编码、路径的构建等，其实本质就是对象的序列化与反序列化的过程。编码就是将 Spring 框架中管理的 Bean 对象序列化为 JSON 格式的数据，方便后面在传输层进行传输，反之，解码就是将接收到的 JSON 字符串解析为框架所管理的 Bean 对象的过程，如图 3.15 是 Bean 编码和解码的过程。

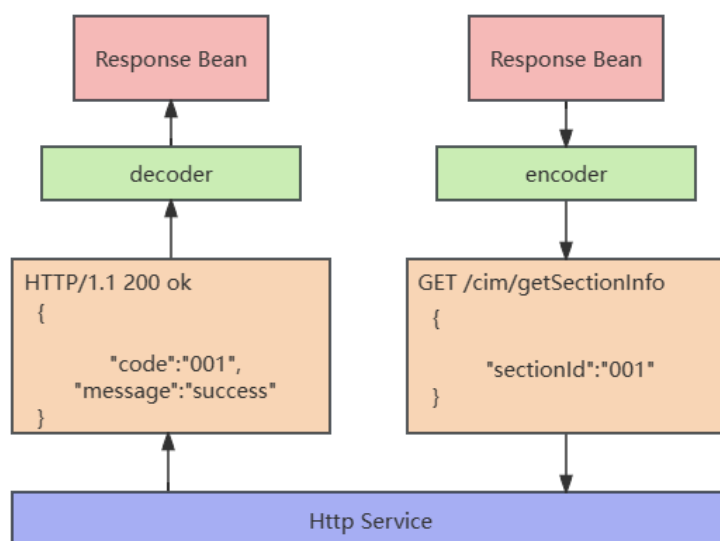


图 3.15 Bean 和 HTTP 请求编解码转化的过程

#### (5) 通信层

这一层是实际完成远程服务通信的一层，基于 HTTP 协议来进行远程通信。通过使用底层的 HTTP 客户端 OkHttpClient 来发送请求和接收响应。在这一层中，需要创建一个 OkHttpClient 实例。OkHttpClient 是 OkHttp 提供的主要类，它封装了发送请求和接收

收响应的核心功能。可以在应用程序中共享一个 `OkHttpClient` 实例，以便复用连接池等资源。接下来，需要创建一个 `Request` 对象，用于描述要发送的 HTTP 请求。`Request` 对象包含了请求的 URL、请求方法（如 GET、POST 等）、请求头、请求体等信息。使用 `OkHttpClient` 实例的 `newCall` 方法创建一个 `Call` 对象，并将 `Request` 对象作为参数传入。然后调用 `Call` 对象的 `execute` 方法发送请求，并阻塞当前线程等待响应。一旦服务器响应到达，`execute` 方法将返回一个 `Response` 对象，其中包含了响应的状态码、响应头、响应体等信息。可以从 `Response` 对象中获取这些信息，并根据需要进行处理。最后，需要调用 `Response` 对象的 `close` 方法释放资源，包括底层的 TCP 连接和响应体。

### 3.3 基于 AHP-熵权法的资源调度优化算法 MMRSA

#### 3.3.1 Kubernetes 默认资源调度算法及问题分析

Kubernetes 默认的资源调度算法是最小资源调度算法，该算法在进行节点选择的时候，会将 CPU 和内存的资源利用率作为关键的参考指标，将 Pod 调度到两种资源利用率最低的节点上。整个调度过程可以分为预选阶段和优选阶段，在预选阶段，Kubernetes 使用一系列的断言对每个节点进行评估。这些断言是关于节点是否满足 Pod 的基本要求，比如 CPU 和内存的资源需求，存储卷的可用性等。如果节点满足了 Pod 的要求，就可以作为候选节点进入到优选阶段。在优选阶段，调度器会基于每一个候选节点的 CPU 和内存资源利用率对每个节点进行打分，并选择得分最高的节点作为最终的 Pod 承载节点进入节点绑定阶段。在这个调度算法当中，存在一些不足如下：

（1）由于 Kubernetes 默认的资源调度算法中只将 CPU 和内存资源的使用情况作为筛选和打分依据，在本项目中部分服务涉及到大量的数据采集或者数据传输、需要占用很高的网络带宽和磁盘 I/O，Kubernetes 的默认资源调度算法在这种场景下不能做出合理的资源调度。

（2）默认的资源调度算法在进行优选阶段打分过程中，CPU 和内存资源利用率得分所占权重是相同的，当需要考虑更多维度因素进行打分时，这种权重分配方式将不再适用。

针对以上不足之处，本文对默认资源调度算法进行了优化，首先在考虑 CPU 和内存资源利用率的基础之上，额外考虑网络带宽以及磁盘 I/O 的资源利用率，结合以上四个节点资源使用情况的重要依据进行筛选和打分，为了更加合理的分配不同指标的权

重占比，本文采用 AHP 结合熵权法的权重分析方法，确定每个指标的对应权重，通过优化的多指标资源调度算法（Multi-Metric Resource Scheduling Algorithm, MMRSA）确定最终的 Pod 承载节点。

### 3.3.2 AHP 权重分析法

AHP 是一种复杂决策的定量分析方法，由美国运筹学家 Thomas L. Saaty 在 20 世纪 70 年代初期提出。AHP 通过将复杂的决策问题分解为多个层次（目标、准则、方案等），再对各层次中的元素进行两两比较，从而确定其相对重要性的过程。该方法结合定性与定量分析，适用于那些难以直接量化的决策问题。

首先基于层次分析法为候选节点多维度的资源指标计算权重值，该权重值将用于后续 MMRSA 算法为节点进行打分。权重值计算过程如下：

（1）构建层次分析模型，该模型一共分为三层，分别为目标层、准则层和方案层，如图 3.16 所示，在本模型中，目标层即为选出最优的候选节点，准则层一共包含四个因素，即 CPU、内存、网络带宽、磁盘 I/O，方案层就是所有的候选节点。

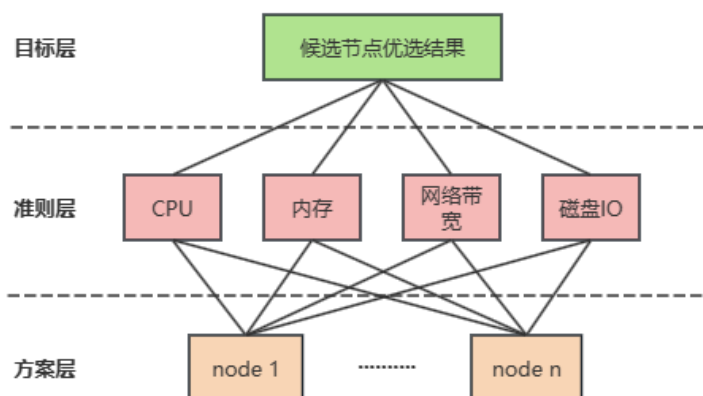


图 3.16 层次分析模型

（2）构建判断矩阵，该判断矩阵能够体现上述四个因素在整个算法中的重要程度。AHP 算法的重要程度是通过两两比较来量化的，判断矩阵当中， $a_{ij}$  代表第  $i$  个指标相较于第  $j$  个指标的重要程度。本文定义了两个因素不同重要程度差距下的标度信息。如表 3.1 所示。

表 3.1 层次分析法标度表

| 标度 $a_{ij}$      | 含义                          |
|------------------|-----------------------------|
| 1                | 表示因素 $j$ 比因素 $i$ 重要性相同      |
| 3                | 表示因素 $j$ 比因素 $i$ 稍微重要       |
| 5                | 表示因素 $j$ 比因素 $i$ 明显重要       |
| 7                | 表示因素 $j$ 比因素 $i$ 极端重要       |
| 1, 1/3, 1/5, 1/7 | 表示比较的主次和上述几种标度相反，重要程度跟分母相对应 |

在本电力营销系统当中，CPU、内存、网络带宽、磁盘 I/O 重要程度逐级递减。所以 CPU 和内存的标度设置为 3、和网络带宽的标度设置为 5，和磁盘 I/O 的标度设置为 7。内存和网络带宽的标度设置为 3，内存和磁盘 I/O 的标度设置为 5，网络带宽和磁盘 I/O 的标度设置为 3。根据设置的标度值，可以构造出的判断矩阵 A。

$$A=\begin{bmatrix} 1 & 3 & 5 & 7 \\ 1/3 & 1 & 3 & 5 \\ 1/5 & 1/3 & 1 & 3 \\ 1/7 & 1/5 & 1/3 & 1 \end{bmatrix}$$

（3）由于判断矩阵的构造会基于个人偏好，存在一定的主观性，为了确保决策过程的逻辑性和合理性，在构造好判断矩阵之后，需要对判断矩阵进行一致性检验，一致性检验的公式如（3-1）、（3-2）所示：

$$CI = \frac{\lambda_{\max} - n}{n - 1} \tag{3-1}$$

$$CR = \frac{CI}{RI} \tag{3-2}$$

式中：CI 代表一致性指标，RI 代表随机一致性指标，n 代表矩阵的阶数，也就是权重因素的个数，λ<sub>max</sub> 代表最大的特征值，CR 代表一致性比率

RI 是预先计算好的，用于不同维度（即矩阵的阶数 n）的标准一致性指数，这些值是基于随机生成的判断矩阵的一致性比率的平均值。对于维度从 1 到 10 的矩阵，RI 的值如表 3.2 所示。矩阵是否可以通过一致性检验依据的就是计算出来的一致性比率 CR，只有 CR<0.1 的时候矩阵才被认为是可以被接收的。

表 3.2 一致性指标对照表

| n（维度） | RI 值 |
|-------|------|
| 1     | 0.00 |
| 2     | 0.00 |
| 3     | 0.58 |
| 4     | 0.90 |
| 5     | 1.12 |
| 6     | 1.24 |
| 7     | 1.32 |

根据上述公式，可以计算出本判断矩阵的最大特征值、权重向量、一致性指标以及一致性比率如表 3.3 所示：

表 3.3 一致性验证过程数据

| CI     | CR     | λ <sub>max</sub> |
|--------|--------|------------------|
| 0.0390 | 0.0433 | 4.117            |

(4) 经过计算得出结论, 判断矩阵符合要求, 继续计算最终的权重向量, 对上述矩阵归一化后的向量就是权重向量, 最终得到的权重向量  $W_1$ 。

$$W_1 = [0.5650 \quad 0.2622 \quad 0.1175 \quad 0.0553]^T$$

至此, 通过层次分析法算法得到了进行节点打分时所需的各个维度资源的权重值, 并通过一致性检验验证了权重值的合理性。后续会将该权重向量和下一节通过熵权法计算出的权重向量结合计算出最终的权重向量。

### 3.3.3 熵权法权重分析法

熵权法是一种客观赋权方法, 通过计算指标的熵值来确定各指标的权重。熵值越小, 说明该指标的变异性越大, 信息量越大, 因而应赋予较大的权重。熵权法的权重计算过程如图 3.17 所示

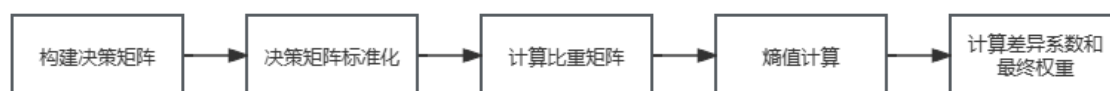


图 3.17 资源调度阶段流程图

熵权法计算权重的流程如下:

(1) 构建决策矩阵, 本文将依据每一个集群节点上的 CPU、内存、网路带宽和磁盘 I/O 四个维度的资源利用率数据来构建一个  $n$  行  $m$  列的决策矩阵  $B$ , 其中,  $n$  代表当前服务器集群中的节点个数,  $m$  代表计算权重的权重指标个数。四个维度的资源利用率通过部署在集群当中的集群监控工具 Prometheus 获取, 表 3.4 是 Prometheus 采集到的集群当中某一时刻的资源利用率情况。

表 3.4 集群中某一时刻的资源利用率

| 节点编号  | 资源利用率 |    |      |        |
|-------|-------|----|------|--------|
|       | CPU   | 内存 | 网络带宽 | 磁盘 I/O |
| Node1 | 21    | 25 | 29   | 33     |
| Node2 | 33    | 28 | 35   | 32     |
| Node3 | 41    | 40 | 30   | 28     |
| Node4 | 46    | 35 | 33   | 36     |
| Node5 | 38    | 37 | 38   | 30     |

根据表 3.4 构建出决策矩阵  $B$ 。

$$B = \begin{bmatrix} 21 & 25 & 29 & 33 \\ 33 & 28 & 35 & 32 \\ 41 & 40 & 30 & 28 \\ 46 & 35 & 33 & 36 \\ 38 & 37 & 38 & 30 \end{bmatrix}$$

(2) 对决策矩阵进行标准化处理, 由于本文选取的四个决策因素都是越低越好的资源利用率, 所以本文采用负向指标的标准化公式对矩阵进行标准化。负向指标的标准化公式如式(3-3)所示:

$$c_{ij} = \frac{\max(b_{ij}) - b_{ij}}{\max(b_{ij}) - \min(b_{ij})} \quad (3-3)$$

式中:  $\max(b_{ij})$  是矩阵第  $j$  列上的最大值,  $\min(b_{ij})$  是矩阵第  $j$  列上的最小值。

根据该负向指标得到标准化矩阵  $C$ 。

$$C = \begin{bmatrix} 1.0000 & 1.0000 & 1.0000 & 0.3750 \\ 0.5200 & 0.8000 & 0.3334 & 0.5000 \\ 0.2000 & 0.0000 & 0.8889 & 1.0000 \\ 0.0000 & 0.3333 & 0.5556 & 0.0000 \\ 0.3200 & 0.2000 & 0.0000 & 0.7500 \end{bmatrix}$$

(3) 数据标准化处理完成后, 得到了每个节点在各指标上的无量纲形式。下一步是计算各节点各指标的比重, 即每个指标在每个节点上的占比。这是为了进一步计算熵值的准备工作。计算第  $i$  个样本值在第  $j$  个指标参数中所占的比重。公式如式(3-4)所示:

$$p_{ij} = \frac{c_{ij}}{\sum_{i=1}^n c_{ij}}, i=1, 2, \dots, n; j=1, 2, \dots, m \quad (3-4)$$

式中:  $p_{ij}$  是第  $i$  个节点在第  $j$  个指标上的比重,  $c_{ij}$  代表第  $i$  个节点在第  $j$  个指标上的标准化值。

(4) 根据比重矩阵计算每一项指标的熵值, 可以通过熵值了解每个指标分布的均匀程度, 进而决定各指标的权重。计算公式如(3-5)所示:

$$E_j = -\frac{1}{\ln(n)} \sum_{i=1}^n p_{ij} \ln(p_{ij}), j=1, 2, \dots, m \quad (3-5)$$

式中:  $E_j$  是第  $j$  个指标的熵值,  $p_{ij}$  是第  $i$  个节点在第  $j$  个指标上的比重,  $n$  是节点的数量。

(5) 计算每一项指标的差异系数, 计算公式如(3-6)所示:

$$G_j = 1 - E_j \quad (3-6)$$

式中:  $G_j$  是差异系数,  $E_j$  是第  $j$  个指标的熵值。

(6) 计算各项指标的最终权重, 计算公式如(3-7)所示:

$$W_2 = \frac{G_j}{\sum_{j=1}^m G_j}, j=1, 2, \dots, m \quad (3-7)$$

通过上述公式得到最终的熵权法权重向量  $W_2$ 。

$$W_2 = [0.2861 \quad 0.2842 \quad 0.2187 \quad 0.2110]^T$$

### 3.3.4 资源调度优化算法 MMRSA

前面两节本文通过权重分析法和熵权法分别计算了四种打分因素对应的权重向量，因为权重分析法掺杂了一些主观的判断，这种主观性会导致权重的计算并不够合理，所以本文结合熵权法这种客观的权重分析方法，将两种方法计算的权重值再次进行加权计算，通过结合熵权法的方式削弱权重分析法中主观性对权重分析带来的影响。经过系统集群环境中的实验验证，将权重分析法的权重设置为 70%，将熵权法的权重设置为 30%可以得到最好的资源调度效果，由此得到最终的权重向量  $W$ 。

至此，得到了进行节点打分时所需的各个维度资源的权重值。接下来对节点的打分算法进行设计。

与 Kubernetes 默认资源调度优选算法相比，本文提出的 MMRSA 算法在 Kubernetes 资源调度器优选阶段会综合考虑 CPU、内存、网络带宽以及磁盘 I/O 等多个维度的节点资源，同时给出更合理的权重分配方案，满足电力营销系统的节点部署调度需求。节点最终得分的计算方案如下：

假设当前节点个数为  $k$ ，节点分别用  $N_1, N_2, \dots, N_k$  表示。接下来定义节点上每种资源的评分计算方法。为了使各项因素计算结果在相同数量级，本文限制每种资源的得分范围在 0 到 100 之间，100 代表最佳适配，0 代表不适配。CPU 得分计算方法如式（3-8）、（3-9）所示：

$$U_i^{cpu} = \frac{C_i^{all} - C_i^{avail}}{C_i^{all}} \quad (3-8)$$

$$S_c = (1 - U_i^{cpu}) * 100 \quad (3-9)$$

式中： $C_i^{avail}$ 、 $C_i^{all}$  是节点  $N_i$  的 CPU 空闲增加量和 CPU 总增加量， $U_i^{cpu}$  代表 CPU 的使用率， $S_c$  代表当前节点 CPU 的得分。

内存的得分计算方式如式（3-10）、（3-11）所示：

$$U_i^{mem} = \frac{M_i^{all} - M_i^{avail}}{M_i^{all}} \quad (3-10)$$

$$S_m = (1 - U_i^{mem}) * 100 \quad (3-11)$$

式中： $M_i^{avail}$ 、 $M_i^{all}$  分别为节点  $N_i$  可用的内存总量和总内存量， $U_i^{mem}$  代表内存的使用率， $S_m$  代表内存资源的最终得分。

网络带宽得分计算方法如式（3-12）、（3-13）、（3-14）所示：

$$U_{rx} = \frac{N_i^{receive}}{N_i^{max}} \quad (3-12)$$

$$U_{tx} = \frac{N_i^{send}}{N_i^{max}} \quad (3-13)$$

$$S_n = 100 - 100 * (U_{rx} + U_{tx}) / 2 \quad (3-14)$$

式中： $N_i^{receive}$ 、 $N_i^{send}$ 、 $N_i^{max}$  分别为节点  $N_i$  网络接收带宽速率、网络发送带宽速率以及网络设备最大带宽， $U_{rx}$  代表接收网络带宽利用率， $U_{tx}$  代表发送网络带宽利用率， $S_n$  代表网络带宽资源的最终得分。

磁盘得分计算方式如式（3-15）、（3-16）、（3-17）所示：

$$U_{read} = \frac{D_i^{read}}{D_i^{max}} \quad (3-15)$$

$$U_{write} = \frac{D_i^{write}}{D_i^{max}} \quad (3-16)$$

$$S_d = 100 - 100 * (U_{read} + U_{write}) / 2 \quad (3-17)$$

式中： $D_i^{read}$ 、 $D_i^{write}$ 、 $D_i^{max}$  分别为节点  $N_i$  磁盘读速率、磁盘写速率以及磁盘最大的读写速率， $U_{read}$  代表磁盘读利用率， $U_{write}$  代表磁盘写利用率， $S_d$  代表磁盘资源的最终得分。

至此  $N_i$  节点的所有打分因素都已经单独打分完毕，结合四大指标的权重来计算最终的节点得分，计算公式如（3-18）所示：

$$S_f = S_c * k_c + S_m * k_m + S_n * k_n + S_d * k_d \quad (3-18)$$

式中： $k_c$ 、 $k_m$ 、 $k_n$ 、 $k_d$  分别代表 CPU、内存、网络带宽以及磁盘 I/O 的权重， $S_f$  代表节点的最终得分。



## 3.4 基于 MMRSA 算法的优化资源调度器需求分析与设计

### 3.4.1 调度器需求分析

为了解决 Kubernetes 在集群内部节点遇到网络和磁盘 I/O 瓶颈时资源调度不合理的问题，本文在上一节设计了一个集群资源调度算法，该算法除了 CPU 和内存资源以外，还将网络带宽和磁盘 I/O 这两个因素考虑到了节点的打分环节当中，并且采用层次分析法和熵权法计算各个因素的权重占比。资源调度器作为资源调度算法的载体，除了调度算法以外，完整的资源调度器在设计阶段还有几个需要解决的问题。首先是资源调度器的方案选择问题，随着 Kubernetes 的不断迭代更新，陆续出现了多种实现定制化资源调度的方式，选择合适的实现方案可以避免资源调度出现冲突，保证集群中服务的稳定运行，还可以降低后期的开发和运维成本。另外由于本文设计的资源调度算法会涉及到更多的集群资源数据，所以需要设计集群中各种数据的获取方案，包括获取 CPU、内存、网络带宽、磁盘 I/O 等资源的近期指标，这些也是需要提前设计考虑的问题。在确定了调度器实现方案之后，还需要基于该方案对调度器进行详细设计。在接下来的几个小节当中，本文会逐个分析以上问题并结合本项目的实际需求和现状确定最终的技术解决方案。

### 3.4.2 调度器方案设计

Kubernetes 的资源调度器 Kube-scheduler 是独立运行在 Kubernetes 当中的进程，其主要功能就是监听从 API Server 获取到的 Pod 信息，然后根据具体的调度算法将对应的 Pod 调度到集群中最合适的节点上。在实际生产应用时，会根据项目的实际特性设计更符合用户应用场景的资源调度器，本文为了解决电力营销系统当中在网络拥塞场景下 Kubernetes 资源调度器资源调度不合理的问题，设计了一个资源调度器。在 Kubernetes 中，定制资源调度器可以采用几种不同的方法，每种方法有其特定的应用场景和优缺点。

直接修改官方 kube-scheduler 的源代码是一种方法。这涉及克隆源代码库，然后在合适的地方添加自定义的调度逻辑。虽然这提供了最大程度的自由度，但这种方法通常不推荐，因为它要求开发者跟踪和合并 Kubernetes 的更新，这会导致维护上的复杂性和额外的工作量。

第二种方法是并行运行一个独立的调度程序，与默认的调度器一起工作。这种方法允许定制调度器和默认调度器通过 Pod 的 `spec.schedulerName` 属性来区分它们负责的

Pods。然而，这种方法的挑战在于，当多个调度器存在时，会出现资源竞争或冲突，例如两个调度器会将不同的 Pod 调度到同一个资源受限的节点上。

第三种方法是使用调度器扩展程序，它本质上是一个可配置的 Webhook，包括过滤器和优先级端点，对应于调度周期中的过滤和打分阶段。然而，这种方法在 Kubernetes v1.16 版本中已被废弃，显示了 Kubernetes 生态系统中技术的快速发展和变化。

第四种方法，即通过调度框架 Scheduling Framework 是目前 Kubernetes 的主流自定义调度方案。这个框架在 Kubernetes v1.15 版本中引入，它为现有调度器添加了一组插件化的 API，使得定制调度器变得更加容易。这些 API 允许在保持调度核心的简单性和易于维护的同时，实现大多数调度功能作为插件的形式。这种方法的优势在于它提供了灵活性和可维护性，使得开发者可以更容易地添加或修改调度逻辑，而不必担心底层调度器的复杂性。

总的来说，如表 3.5 所示，尽管有多种方法可以定制 Kubernetes 的资源调度器，但通过调度框架的方式因其插件化、灵活性和与 Kubernetes 紧密集成的特点而成为首选方法。它简化了定制调度器的创建和维护过程，使得调度器能够更好地适应不断变化的需求和 Kubernetes 生态系统的发展。因此本文将采用方案四的方式完成资源调度器的定制开发。

表 3.5 四种调度器设计方案对比

| 方法                            | 描述                                                              | 优点                                                 | 缺点                                     |
|-------------------------------|-----------------------------------------------------------------|----------------------------------------------------|----------------------------------------|
| 直接修改官方 kube-scheduler 的源代码    | 通过修改 Kubernetes 的默认调度器 kube-scheduler 的源代码来实现定制调度逻辑             | 完全控制调度逻辑，可以实现任何级别的定制                               | 升级和维护难度大，需要深入了解 kube-scheduler 的内部工作机制 |
| 并行运行一个独立的调度程序，与默认的调度器一起工作     | 启动一个或多个独立的调度器实例，这些实例并行于默认的 kube-scheduler 运行，并且可以根据需要调度特定的 Pod  | 灵活性高，可以并行运行多个调度器，不需要修改默认调度器的代码                     | 管理复杂度增加，需要处理调度冲突                       |
| 使用调度器扩展程序，它本质上是一个可配置的 Webhook | 通过编写一个 Webhook 服务来影响调度决策。在调度过程中，kube-scheduler 会调用这个 Webhook 服务 | 可以较为容易地插入自定义逻辑                                     | 与调度器的通信增加了延迟，需要维护额外的 Webhook 服务        |
| 通过调度框架（Scheduling Framework）  | 使用 Kubernetes 调度框架插件机制，通过编写插件来定制调度过程                            | 具有插件化、灵活性的特点，并且与 Kubernetes 紧密集成，简化了自定义调度器的创建和维护过程 | 需要了解调度框架的插件机制                          |

### 3.4.3 调度器资源指标获取方案

由于本资源调度器需要获取集群内节点实时的内存、CPU 以及网络带宽等数据，就需要一个获取资源的集群监控平台，本文选用的集群监控方案为 Prometheus，Prometheus 是一个功能强大的监控系统，支持多种类型的指标收集和度量，包括计数器、直方图等。这种多样性使得它能够捕获各种与性能相关的数据，例如延迟、吞吐量、错误率等。同时 Prometheus 具有灵活而强大的查询语言 PromQL，可以执行复杂的数据查询和分析操作。这使得用户能够根据自己的需求轻松地创建自定义的性能监控指标和仪表盘。最关键的是，Istio 与 Prometheus 集成得非常紧密，因此它可以无缝地与 Istio 的监控功能集成。在 Istio 环境下使用 Prometheus 相对其他集群监控工具要方便很多。Prometheus 还可以集成 Grafana，Grafana 将 Prometheus 的数据可视化，更直观的展示数据走势。

本文设计的调度器通过 Prometheus 提供的 API 接口来进行监控数据的获取，调度器通过访问 Prometheus Server 来进行数据的查询并且拉取数据。通过不同的 PromQL 查询本文所需要的计算 CPU、内存、网络带宽等资源使用率的数据。设计的具体的 PromQL 语句如下：

CPU:

```
increase(node_cpu_seconds_total{kubernetes_node=\"%s\"}[5m])
```

```
increase(node_cpu_seconds_total{kubernetes_node=\"%s\", mode=\"idle\"}[5m])
```

这两个语句分别用于查询 5 分钟内 CPU 时间总的增加量以及 5 分钟内 CPU 空闲时间的总增加量，{kubernetes\_node=\"%s\", mode=\"idle\"}：这是查询的标签选择器（label selector），用于筛选特定的 Kubernetes 节点和 CPU 模式为“空闲”（idle）的情况。%s 是占位符，在调度器算法实际执行时将会替换为真实的参数值，increase() 函数计算在给定的时间范围内指标值的增加量。

内存:

```
node_memory_MemAvailable_bytes{kubernetes_node=\"%s\"}
```

```
node_memory_MemTotal_bytes{kubernetes_node=\"%s\"}
```

这两个语句分别是查询某个节点的总内存总量以及节点的可用内存总量。{kubernetes\_node=\"%s\"} 是一个标签选择器，用于指定查询的是哪个 Kubernetes 节点的内存总量。这里的 %s 是一个占位符，需要替换为实际节点的名称。

网络带宽:

```
rate(node_network_receive_bytes_total{kubernetes_node="%s",device="%s"}[5m])  
rate(node_network_transmit_bytes_total{kubernetes_node="%s",device="%s"}[5m])
```

如上两个语句分别查询的是某个节点某个网络设备在 5 分钟以内的接收字节的速率和发送字节的速率。其中{kubernetes\_node="%s",device="%s"}是一个标签选择器，用于指定要统计哪个节点的哪个网络设备接口。%s 是一个占位符，在实际的算法执行当中会被实际数据指标替代。

磁盘 I/O:

```
rate(node_disk_read_bytes_total{kubernetes_node="%s"}[5m])  
rate(node_disk_written_bytes_total{kubernetes_node="%s"}[5m])
```

如上两个语句分别是查询 5 分钟以内，磁盘读取和写入字节的速率。{kubernetes\_node="%s"} 是一个标签选择器，用于指定查询的是哪个 Kubernetes 节点的磁盘读写速率。这里的 %s 是一个占位符，需要替换为实际节点的名称。

在数据获取层面，还有两个指标是无法通过 Prometheus 直接获取到的，一个是当前节点的最大网络带宽，另一个是当前节点的最大磁盘写速率。这两个数据是跟服务器节点硬件直接相关的，而不是能够通过 Prometheus 监控出来的，为了解决这个问题，本文采用的方式是通过 Kubernetes 给节点打上带有这两个关键信息的标签，这样调度算法在实现资源调度的时候，可以在决策打分的时候直接通过标签来查询获取这两个信息，标签标记数据的语句格式如下：

```
Kubectl label nodes nodex max-network-bandwidth=xGbps max-disk-io=xMB/s
```

### 3.4.4 调度器详细设计

本文基于调度框架 Scheduling Framework 来实现自定义的资源调度，本调度器是基于预选和优选两个阶段的框架，如图 3.18 所示，预选阶段的目的是快速排除那些不符合基本要求的节点，以减少需要进一步考虑的节点数量。在这个阶段，调度器会执行预置的多个 Filter 插件，只有满足所有 Filter 插件的过滤条件，才能够进入优选阶段。优选阶段的目的是对剩余的节点进行打分，以找到最适合 Pod 的节点。每个节点都会根据一系列的打分算法得到一个分数，调度器会选择分数最高的节点来部署 Pod。而在这个阶段，最关键的就是 Score 打分算法的实现，打分算法的具体实现将直接影响最终资源调度的结果。

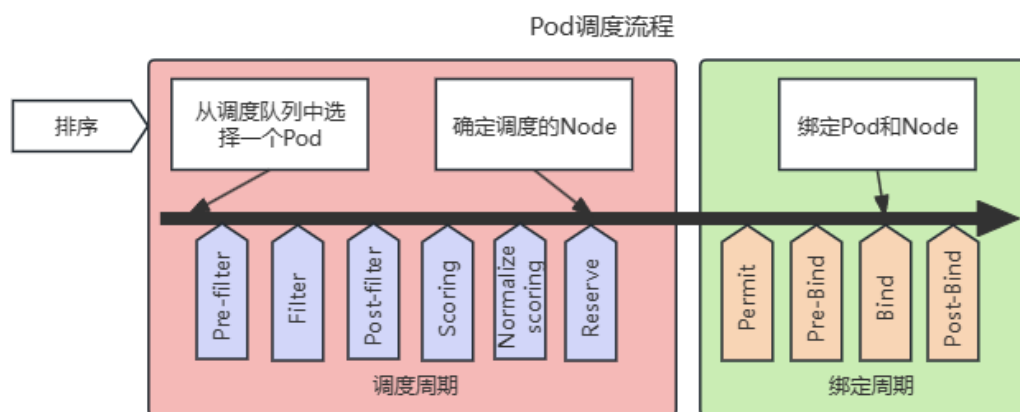


图 3.18 资源调度阶段流程图

实现自定义的资源调度算法，核心任务是实现优选阶段的自定义的 Score 算法，也就是对 Score 扩展点进行扩展，首先是定义一个 Score 插件接口的结构体。这个结构体可以包含你需要的任何状态或配置信息。其次是实现 Score 插件接口的方法，在本调度器当中就是根据节点的 CPU、内存、网络带宽以及磁盘 I/O 的资源可用率对节点进行打分的具体算法实现。

插件实现完成后，还需要注册该自定义插件，这样才可以让调度器识别到对应的调度算法，并在后续的声明式文件中指定调度算法，编码完成后将调度器模块封装打包，并制作成 Docker 镜像，通过声明式 yaml 文件将调度器部署到 Kubernetes 集群当中。

### 3.5 本章小结

本章对原本基于微服务架构的电力营销系统进行了重构，将他的后端代码重构到了非侵入式的服务网格架构体系中。解决了服务治理代码和业务代码耦合度高，系统复杂，中间件版本碎片化的问题。为了更加适配 Istio，本章设计并实现了一个轻量化的服务通信组件，本组件摒弃了开源组件中服务网格不需要的功能点，如负载均衡、熔断等，同时保持了旧有的编码习惯，方便用户上手，使组件更加轻量化。由于本系统的服务网格架构的底层容器编排工具选用的是 Kubernetes，为了解决 Kubernetes 在本系统中资源调度不合理的问题，本文提出了基于 AHP-熵权法的多指标资源调度算法 MM RSA，本资源调度算法对比 Kubernetes 的默认资源调度算法额外考虑了集群节点实时的网络带宽和磁盘 I/O 利用率，同时设计了更加合理的多指标权重分配算法，本调度算法可以实现更加均衡的资源调度。

## 第4章 电力营销系统构建与实验分析

首先,为了进行本章的实验,需要搭建基于 Kubernetes 的 Istio 服务网格云平台,然后本章将从各业务模块轻量化程度、系统多语言可扩展性、集群监控支持、流量管理灵活性等方面对重构后的系统进行优化分析。然后对本文实现的定制服务通信组件进行功能性和非功能性测试。最后对本文提出的基于 AHP-熵权法的多指标资源调度算法 MMRSA 进行资源调度均衡度实验与分析。

### 4.1 服务网格基础设施构建

为了部署本服务网格架构下的电力营销系统,需要在多台服务器上先部署 Kubernetes 集群,并且基于 Kubernetes 集群继续部署 Istio 服务网格,然后将对应的电力营销系统当中的微服务以 pod 的形式部署到服务网格当中。

表 4.1 部署环境信息表

| 设备或软件           | 详细信息                                                                                                                                                                                |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 服务器硬件资源信息 (5 台) | Master 节点 (1 台):8 核 16G<br>网络带宽 20 Mbps<br>磁盘大小 500G<br>Worker 节点 (4 台):8 核 16G<br>网络带宽 20 Mbps<br>磁盘大小 500G                                                                        |
| 操作系统            | CentOS 7.9                                                                                                                                                                          |
| 集群组件版本          | Docker 20.10.7<br>Go 1.15.14<br>Kubernetes 1.20.9<br>Harbor v2.6.0<br>Docker-compose v2.16.0<br>Istio 1.11.2<br>Calico v3.20.6<br>Grafana 7.5.5<br>Kiali v1.38<br>Prometheus 14.3.0 |

如表 4.1 所示,系统测试环境中使用的五台服务器均是 Centos7.9 操作系统的服务器,分别作为集群的 Master 节点和 Node 节点,不包含污点。选用的 Docker 版本为 20.10.7, Kubernetes 的版本为 1.20.9,在对 Kubernetes 进行二次插件开发的过程中使用的 go 语言版本为 go1.15.14,如图 4.1 所示。部署在 Kubernetes 集群上的服务网格 Istio 控制平面和数据平面的版本均为 1.11.2,后续的服务部署与系统测试都将基于这套集群基础设施来进行。

```
[root@istio-master ~]# kubectl get pods -n kube-system
```

| NAME                                     | READY | STATUS  | RESTARTS | AGE  |
|------------------------------------------|-------|---------|----------|------|
| calico-kube-controllers-577f77cb5c-jfl49 | 1/1   | Running | 11       | 126d |
| calico-node-9rw9c                        | 1/1   | Running | 3        | 125d |
| calico-node-gd7q5                        | 1/1   | Running | 1        | 125d |
| calico-node-r4xp7                        | 1/1   | Running | 1        | 125d |
| coredns-5897cd56c4-gjnrc                 | 1/1   | Running | 2        | 150d |
| coredns-5897cd56c4-x7xpx                 | 1/1   | Running | 2        | 150d |
| etcd-istio-master                        | 1/1   | Running | 4        | 150d |
| kube-apiserver-istio-master              | 1/1   | Running | 5        | 150d |
| kube-controller-manager-istio-master     | 1/1   | Running | 4        | 150d |
| kube-proxy-mm9rs                         | 1/1   | Running | 5        | 150d |
| kube-proxy-qrbcm                         | 1/1   | Running | 2        | 150d |
| kube-proxy-srpct                         | 1/1   | Running | 2        | 150d |
| kube-scheduler-istio-master              | 1/1   | Running | 0        | 96d  |

图 4.1 Kubernetes 集群资源部署图

由于本文自定义资源调度器需要持续获取集群节点中的各项指标参数，所以需要  
在服务网络上部署集群监控组件 Prometheus，在完成节点数据采集的同时，还可以负  
责监控整个服务网络集群，还部署了 Grafana、Jaeger 以及 kiali 等组件，如图 4.2 所示，  
用于实现集群中的分布式追踪和集群数据指标度量可视化。

```
[root@istio-master ~]# kubectl get pods -n istio-system
```

| NAME                                  | READY | STATUS  | RESTARTS | AGE  |
|---------------------------------------|-------|---------|----------|------|
| grafana-556f8998cd-2m8fx              | 1/1   | Running | 2        | 148d |
| istio-egressgateway-6c9486d667-96bbr  | 1/1   | Running | 2        | 148d |
| istio-ingressgateway-556bd8b675-sfx2q | 1/1   | Running | 2        | 148d |
| istiod-96b47b576-zt4qz                | 1/1   | Running | 2        | 148d |
| jaeger-5f65fdbf9b-mwd76               | 1/1   | Running | 2        | 148d |
| kiali-787bc487b7-m69cl                | 1/1   | Running | 2        | 148d |
| prometheus-9f4947649-sgptc            | 2/2   | Running | 6        | 148d |

图 4.2 Istio 部署图

## 4.2 云原生架构系统构建与相关优化分析测试

上一节完成了项目部署所需的基础设施环境 Kubernetes 集群和服务网格 Istio 的构  
建，接下来需要进行项目中非业务组件的部署，包括 Minio、MySQL、MongoDB、  
Harbor、Redis 等，这些中间件为项目的业务模块提供技术支持。为了保障整个系统  
的高可用性，除了保证业务模块的稳定，上述中间件的稳定性同样需要保障，本文通  
过构建对应组件集群以及依靠 Kubernetes 管理组件生命周期的方式保证中间件的高可  
用性。

因为服务治理是治理项目中的所有业务模块，流经这些中间件的流量一般不需要  
进行监控管理，所以并不需要为这几个中间件添加网格代理。图 4.3、图 4.4、图 4.5、  
图 4.6 是这些中间件集群部署后的集群状态图或可视化界面图。

```

Source_User: root
Source_Port: 3306
Connect_Retry: 60
Source_Log_File: mysql-bin.000004
Read_Source_Log_Pos: 3455
Relay_Log_File: cfef0507d627-relay-bin.000005
Relay_Log_Pos: 3670
Relay_Source_Log_File: mysql-bin.000004
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Replicate_Do_DB:
Replicate_Ignore_DB: mysql,sys,information_schema,performance_schema
Replicate_Do_Table:
Replicate_Ignore_Table:

```

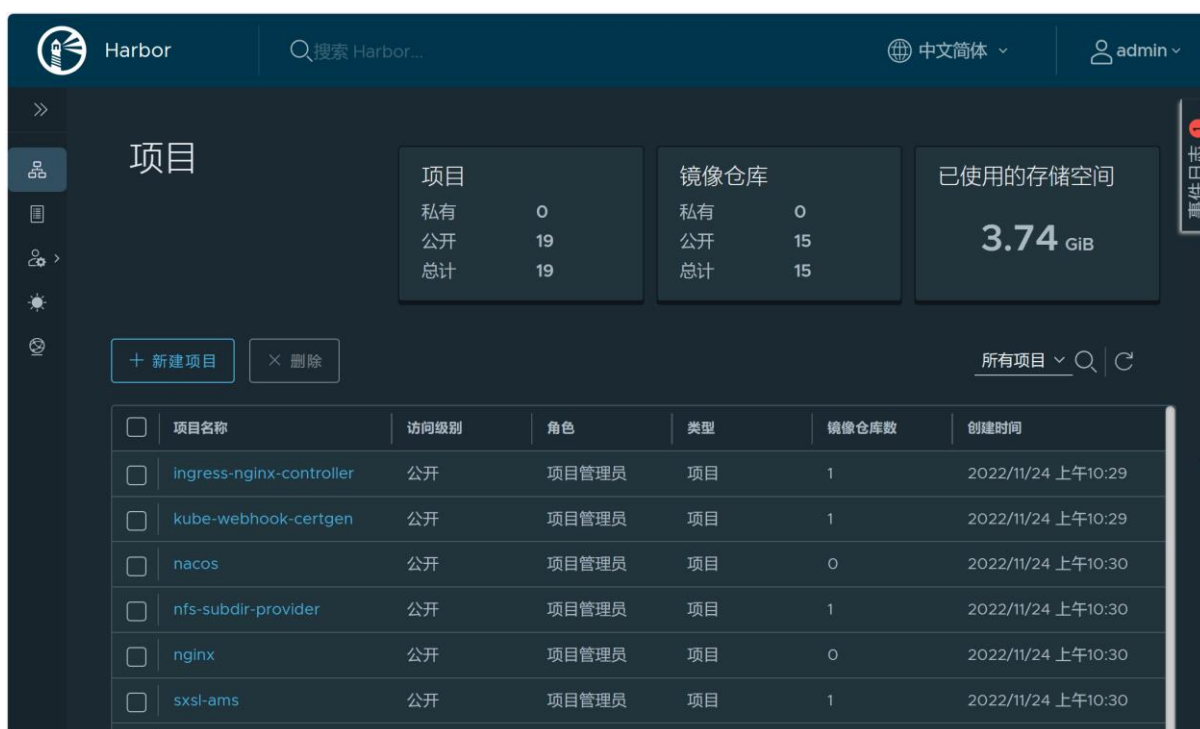
图 4.3 MySQL 主备库部署成功状态图

```

redis-sxs1-33(172.21.29.140:6378)>cluster nodes
"
4c719a3a132f15f4d015ac2267181890b7f5081f 172.21.29.138:6381@16381 master - 0 1664509784520 3 connected 10923-16383
e061e70d69823f4264b9f35377c9fb15066679ec 172.21.29.138:6384@16384 slave 4c719a3a132f15f4d015ac2267181890b7f5081f 0 1664509783000 3
connected
20200e543fa1906419ac21730204378ecfa69f15 172.21.29.140:6382@16382 slave e0ce3c6eca06db033a386975adc80f5f3c17fc45 0 1664509783000 1
connected
e0ce3c6eca06db033a386975adc80f5f3c17fc45 172.21.29.140:6378@16378 myself,master - 0 1664509783000 1 connected 0-5460
5d8ff7b31d1147308e14f80238bf840a46596f49 172.21.29.139:6380@16380 master - 0 1664509783516 2 connected 5461-10922
fcd6a934cee0a3b3135afd631901ad84cd6de02e 172.21.29.139:6383@16383 slave 5d8ff7b31d1147308e14f80238bf840a46596f49 0 1664509783000 2
connected
"

```

图 4.4 Redis 三主三从集群状态图



| 项目 | 私有 | 公开 | 总计 |
|----|----|----|----|
| 项目 | 0  | 19 | 19 |

| 镜像仓库 | 私有 | 公开 | 总计 |
|------|----|----|----|
| 镜像仓库 | 0  | 15 | 15 |

已使用的存储空间: 3.74 GiB

| 项目名称                     | 访问级别 | 角色    | 类型 | 镜像仓库数 | 创建时间               |
|--------------------------|------|-------|----|-------|--------------------|
| ingress-nginx-controller | 公开   | 项目管理员 | 项目 | 1     | 2022/11/24 上午10:29 |
| kube-webhook-certgen     | 公开   | 项目管理员 | 项目 | 1     | 2022/11/24 上午10:29 |
| nacos                    | 公开   | 项目管理员 | 项目 | 0     | 2022/11/24 上午10:30 |
| nfs-subdir-provider      | 公开   | 项目管理员 | 项目 | 1     | 2022/11/24 上午10:30 |
| nginx                    | 公开   | 项目管理员 | 项目 | 0     | 2022/11/24 上午10:30 |
| sxs1-ams                 | 公开   | 项目管理员 | 项目 | 1     | 2022/11/24 上午10:30 |

图 4.5 Harbor 镜像仓库可视化界面



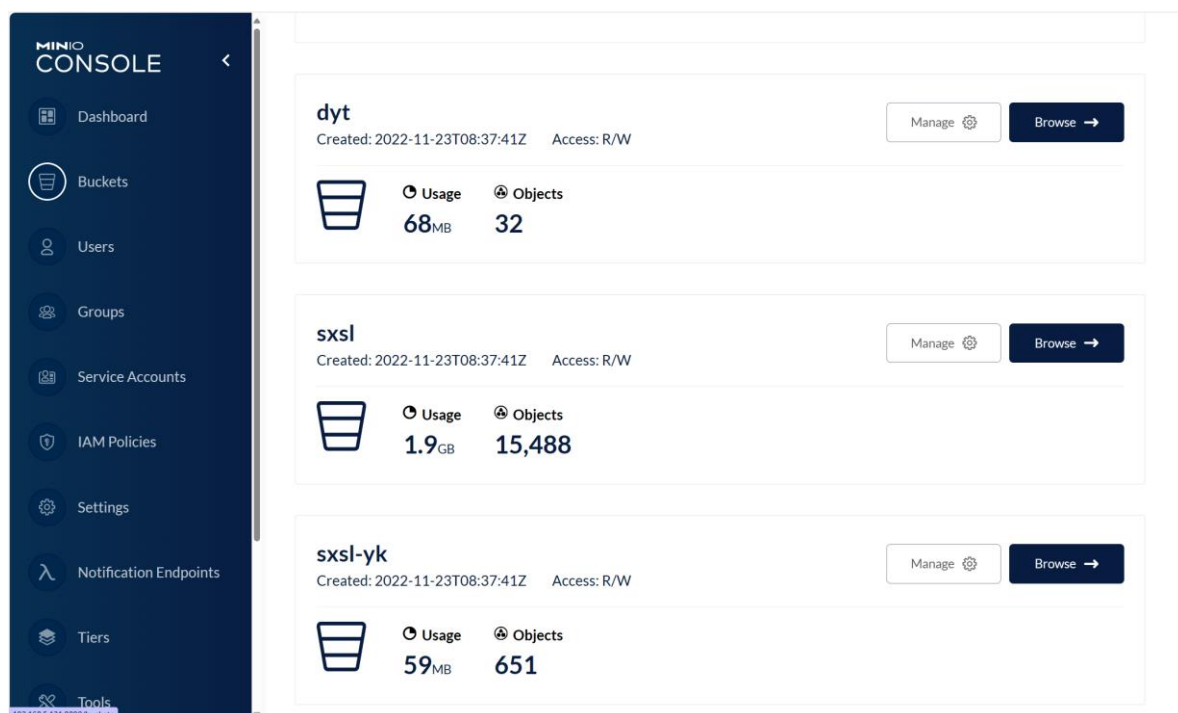


图 4.6 Minio 可视化界面

部署完启动业务服务需要的基础中间件之后，进行系统中各个业务模块的部署。由于业务模块需要受到服务网络的治理与监控，所以需要通过网络为每个业务模块创建对应的代理，如图 4.7 是各个服务部署完成的效果图。图中可以看出，READY 状态处的容器都是两个，这其实不是创建了两个相同的服务容器，其中一个是为服务网络为每一个微服务创建的网格代理的容器，这个代理将负责管理和控制所有进出业务容器的流量，通过控制平面统一下发管理规则到各个代理上，每个代理按照规则进行流量的过滤，同时还会将数据同步给集群监控工具 Prometheus，再通过 Grafana 以可视化的方式展示给运维人员。

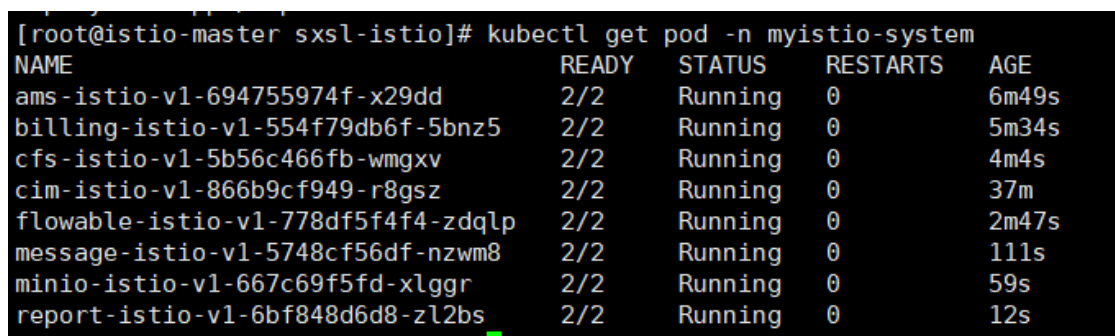


图 4.7 业务服务部署状态图

至此基于云原生架构的电力营销系统已经重构并部署完毕，图 4.8、4.9、4.10 是系统的部分模块界面展示。接下来本文将从各业务模块轻量化程度、系统多语言可扩展性、集群监控支持、流量管理灵活性等方面对重构后的系统优化进行分析。

## 第 4 章 电力营销系统构建与实验分析

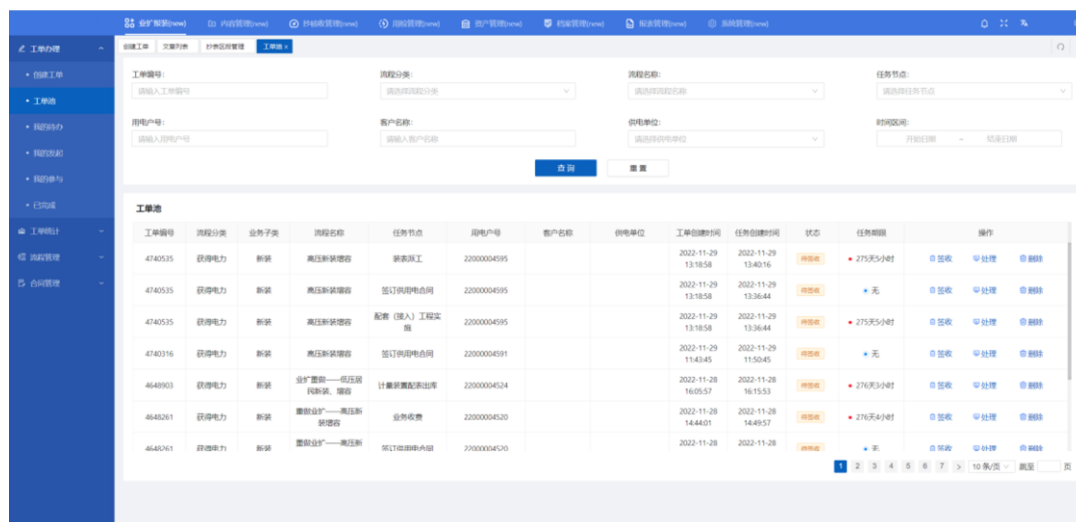


图 4.8 业务扩展模块

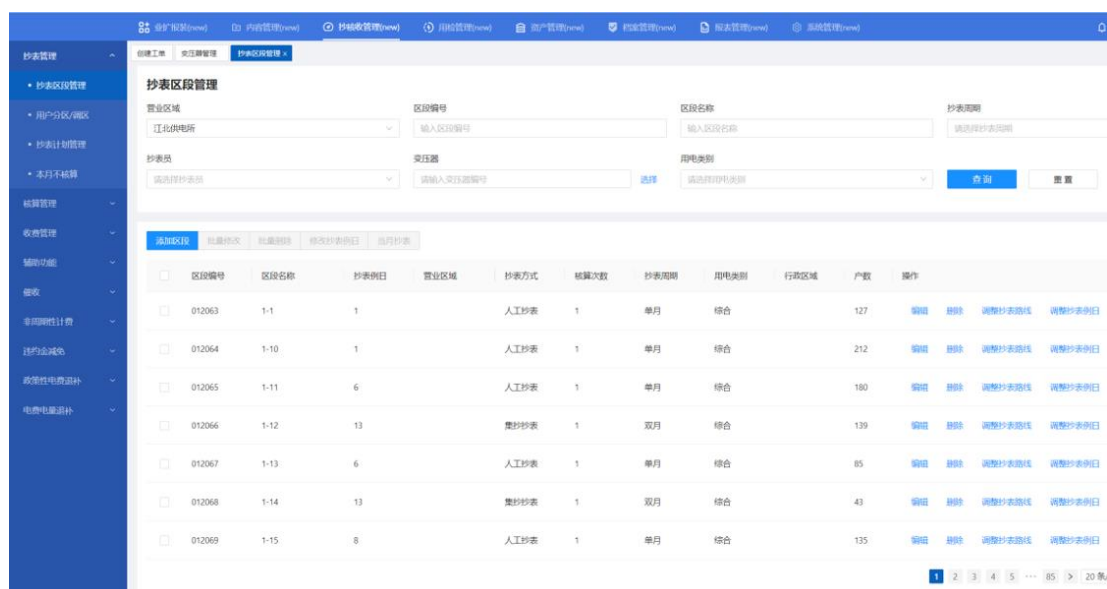


图 4.9 抄表核验收款模块

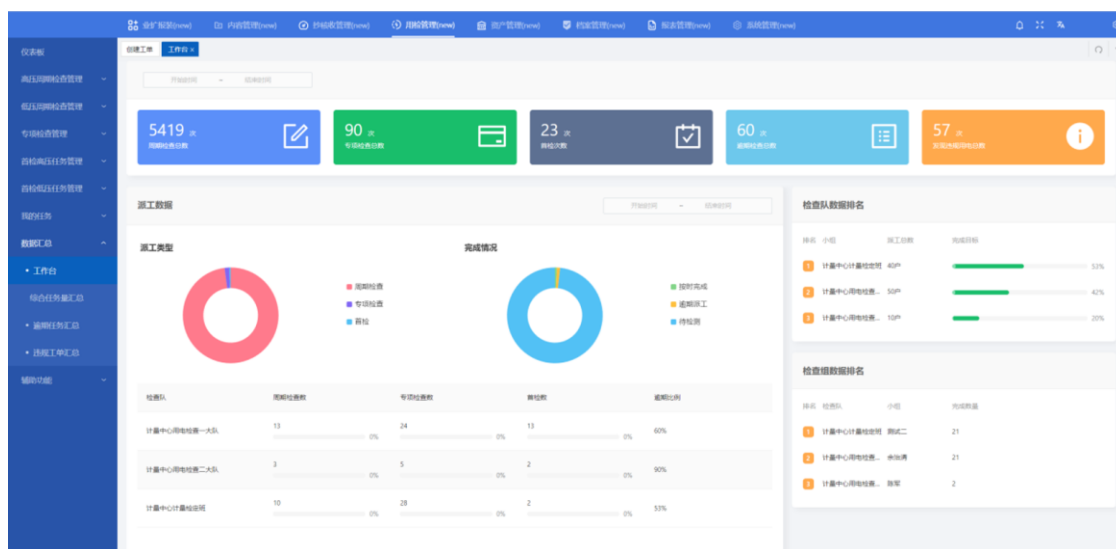


图 4.10 用检模块

由于在项目重构的过程中，本文将侵入式的服务治理方式重构为非侵入式的服务治理方式，将侵入式的服务治理相关的 SDK 都剥离出项目模块，使各个模块更加轻量化，所以项目中各个微服务模块在打包文件大小上会有一定程度的优化，接下来对这个优化程度做量化分析。

如图 4.11 所示，是项目各个模块重构前后的模块大小对比图，可以看出，各个模块在剥离侵入式的服务治理组件之后，模块大小都有不同程度的优化，整理图 4.11 中的数据形成表 4.2。

|                                                                                                                       |              |              |            |                                                                                                                       |              |              |            |
|-----------------------------------------------------------------------------------------------------------------------|--------------|--------------|------------|-----------------------------------------------------------------------------------------------------------------------|--------------|--------------|------------|
|  billing-old-0.0.1-SNAPSHOT.jar      | 2024/3/4 ... | Executabl... | 94,503 KB  |  billing-new-0.0.1-SNAPSHOT.jar      | 2024/3/4 ... | Executabl... | 89,021 KB  |
|  cim-old-0.0.1-SNAPSHOT.jar          | 2024/2/2...  | Executabl... | 103,883 KB |  cim-new-0.0.1-SNAPSHOT.jar          | 2024/3/4 ... | Executabl... | 97,972 KB  |
|  ams-old-0.0.1-SNAPSHOT.jar          | 2022/11/...  | Executabl... | 74,477 KB  |  ams-new-0.0.1-SNAPSHOT.jar          | 2024/3/5 ... | Executabl... | 71,979 KB  |
|  cfs-old-0.0.3-SNAPSHOT.jar          | 2024/3/4 ... | Executabl... | 40,846 KB  |  cfs-new-0.0.3-SNAPSHOT.jar          | 2024/3/5 ... | Executabl... | 38,198 KB  |
|  elec-old-0.0.1-SNAPSHOT.jar         | 2022/11/...  | Executabl... | 59,765 KB  |  elec-new-0.0.1-SNAPSHOT.jar         | 2024/3/5 ... | Executabl... | 57,870 KB  |
|  flowable-app-old-1.0.0-SNAPSHOT.jar | 2022/11/...  | Executabl... | 188,709 KB |  flowable-app-new-1.0.0-SNAPSHOT.jar | 2024/3/4 ... | Executabl... | 181,264 KB |
|  message-old-1.0.0-SNAPSHOT.jar      | 2022/11/...  | Executabl... | 147,980 KB |  message-new-1.0.0-SNAPSHOT.jar      | 2024/3/4 ... | Executabl... | 139,691 KB |
|  minio-old-1.0.0-SNAPSHOT.jar        | 2022/11/...  | Executabl... | 72,098 KB  |  minio-new-1.0.0-SNAPSHOT.jar        | 2024/3/4 ... | Executabl... | 70,412 KB  |
|  report-old-0.0.1-SNAPSHOT.jar       | 2024/3/4 ... | Executabl... | 91,803 KB  |  report-new-0.0.1-SNAPSHOT.jar       | 2024/3/4 ... | Executabl... | 85,314 KB  |
|  security-old-0.0.1-SNAPSHOT.jar    | 2024/3/4 ... | Executabl... | 95,861 KB  |  security-new-0.0.1-SNAPSHOT.jar    | 2024/3/4 ... | Executabl... | 93,999 KB  |

图 4.11 项目各模块重构前后文件大小

经计算，得出重构后的模块大小相较于重构之前平均缩小了 4.5%，在一定程度上达到了轻量化微服务模块的效果。

表 4.2 各模块大小对比

| 模块名          | 重构前    | 重构后    | 缩小比例 |
|--------------|--------|--------|------|
| Billing      | 94.5M  | 89.0M  | 5.8% |
| Cim          | 103.8M | 97.9M  | 5.6% |
| Ams          | 74.4M  | 71.9M  | 3.3% |
| Cfs          | 40.8M  | 38.1M  | 6.6% |
| Elec         | 59.7M  | 57.8M  | 3.1% |
| Flowable-app | 188.7M | 181.2M | 3.9% |
| Message      | 147.9M | 139.6M | 5.6% |
| Minio        | 72.0M  | 70.4M  | 2.2% |
| Report       | 91.8M  | 85.3M  | 7%   |
| Security     | 95.8M  | 93.9M  | 1.9% |
| 平均值          |        |        | 4.5% |

重构后的电力营销系统优化了系统各个微服务模块的大小，同时也优化掉了一些微服务架构下的服务中间件，包括 Sentinel、Nacos、Seata。架构的升级摆脱了微服务架构体系下系统对诸多服务治理中间件的依赖，解决了系统的依赖版本碎片化问题。

为了验证服务网格的多语言扩展性，本文在网格中部署了一个 python 应用程序，图 4.12 是部署成功效果图，这在原微服务架构体系下是无法实现的，因为在微服务架构下，使用的服务治理技术是基于 Java 技术栈的，会限制系统中模块语言的多样性。

将项目重构后，整个集群系统的服务治理任务都交给服务网格来完成，解除了模块的语言限制，现在的系统可以将不同语言开发的模块统一接入服务网格来进行服务治理，提高了整个系统的可扩展能力。

```
[root@istio-master sxml-istio]# kubectl get pod -n myistio-system
NAME                                READY   STATUS    RESTARTS   AGE
ams-istio-v1-694755974f-x29dd      2/2     Running   0           17d
billing-istio-v1-554f79db6f-5bnz5  2/2     Running   0           17d
cfs-istio-v1-5b56c466fb-wmgxv     2/2     Running   0           17d
cim-istio-v1-866b9cf949-r8gsz     2/2     Running   0           18d
flowable-istio-v1-778df5f4f4-zdqlp 2/2     Running   0           17d
message-istio-v1-5748cf56df-nzwm8  2/2     Running   0           17d
minio-istio-v1-667c69f5fd-xlggr    2/2     Running   0           17d
python-extension-v1-6c97f99957-xxkzm 2/2     Running   0           9s
report-istio-v1-6bf848d6d8-zl2bs   2/2     Running   0           17d
[root@istio-master sxml-istio]# kubectl get pod -n myistio-system | grep python
python-extension-v1-6c97f99957-xxkzm 2/2     Running   0           34s
```

图 4.12 跨语言支持能力测试

由于本架构中包含集群监控工具 Prometheus 以及其可视化工具 Grafana，Istio 结合 Prometheus 可以实现对整个服务网格的全面监控，包括服务间的通信、流量流向、延迟、错误率等方面的指标，还有集群中服务器节点各项资源的利用率情况。相较于微服务架构，本套框架具有更强的可观测性。如图 4.13 是 Grafana 集群监控可视化界面的部分监控数据。



图 4.13 Grafana 可视化界面部分监控数据

当前基于 Istio 的电力营销系统，能够减轻系统运维负担，提高系统运维效率，例如可以通过灵活的流量控制功能来实现灰度发布、AB 测试等，项目迭代更新发布的过程中，本架构可以提供更好的系统稳定性。如图 4.14 是一个简易的能够根据服务版本

实现流量分配的声明式文件。灰度发布和 AB 测试的实现方式比较相似，本质上都是通过 VirtualService 对请求某个模块的所有流量进行管控和精确分配，这种分配可以是随机的，也可以是根据请求路由等进行正则匹配的结果，可以根据实际需求定制流量分配规则，大大提高了测试的灵活性。

```
[root@istio-master networking]# cat virtual-service-istio-cim.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: istio-cim
spec:
  hosts:
  - istio-cim
  http:
  - route:
    - destination:
        host: istio-cim
        subset: v1
        weight: 80
    - destination:
        host: istio-cim
        subset: v2
        weight: 20
```

图 4.14 实现灰度发布的声明式文件

本服务治理方案还可以轻松的完成集群中对某个模块的故障注入，故障注入主要是为了验证服务的容错能力，演练系统在发生灾难性故障时的恢复方案是否有效。如图 4.15 是故障注入的关键实现案例，在本案例当中，对所有请求 istio-billing 服务 uri 前缀为 backCharge 的接口进行 2s 的延迟注入，注入范围是 100%的请求流量，图 4.16 是故障注入的验证结果，可以看出 2s 延迟注入成功，这在微服务架构中是需要借助网关或者其他组件来辅助完成的，现在服务网格天然支持并且更加灵活且可监控。

```
[root@istio-master networking]# cat gzzr-istio-billing.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: istio-billing-injection
spec:
  hosts:
  - istio-billing
  http:
  - match:
    - uri:
        prefix: "/backCharge"
    fault:
      delay:
        fixedDelay: 2s
        percentage:
          value: 100
    route:
    - destination:
        host: istio-billing
```

图 4.15 故障注入关键配置

```
[root@istio-master networking]# time curl "http://istio-billing/backCharge/getCharge?userNo=32441132"
{"chargeId":"77523123"}
real 0m0.009s
user 0m0.004s
sys 0m0.001s
[root@istio-master networking]# time curl "http://istio-billing/backCharge/getCharge?userNo=32441132"
{"chargeId":"77523123"}
real 0m2.010s
user 0m0.005s
sys 0m0.003s
```

图 4.16 故障注入验证

## 4.3 服务通信组件的测试

### 4.3.1 实验目标

本节的实验目标是对服务通信组件进行功能性和非功能性测试。功能性测试部分主要是验证该组件远程通信能力的正确性，也就是在进行远程调用的过程中是否可以得到正确的响应结果。非功能性测试主要是以当前主流的服务通信组件 OpenFeign 为对照组，测试在高并发应用场景下应用本组件的接口平均响应时间，以及本组件相较于 OpenFeign 的轻量化程度。

### 4.3.2 实验方案

通信组件的功能性测试本文设计了两类测试用例来进行验证，一类是会携带参数的测试用例，一类是未携带参数的测试用例。进行测试时，需要同时准备本地服务以及可以被本地服务调用的远程服务。具体的测试用例如表 4.3 所示，使用远程接口调用工具 Postman 按照表 4.3 对接口进行测试

表 4.3 服务通信组件功能性测试用例

| 测试类型                                                                           | 测试方法                                                            | 预期响应               |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------|
| 未携带数据的远程服务调用<br>GET<br>http://istio-cim:10007/cim/getStationIds                | 1.实现远程调用接口，添加注解以支持远程调用<br>2.启动服务进行远程调用<br>3.接收响应结果进行分析处理        | 返回所有抄表区段信息         |
| 携带了数据的远程服务调用<br>GET<br>http://istio-billing:10008/billing/getBillIds/{userNum} | 1.实现远程调用接口，添加注解以支持远程调用<br>2.启动服务，携带用户信息进行远程调用<br>3.接收响应结果进行分析处理 | 返回该用户名下所有用电户的用电量信息 |

对于通信组件的非功能性测试，测试工具选择 Apache 公司的开源压力测试工具 JMeter，本文选取的对照服务通信组件是 Spring Cloud 框架中的 OpenFeign，也是目前主流的服务通信组件之一。根据该接口的复杂程度与单次请求响应时间，会选取四个不同等级的接口作为性能测试的测试用例，分别为：接口复杂度较低，单次请求耗时在 250ms 以内的接口、接口复杂度中等，单次请求耗时在 250-500ms 内的接口、接口复杂度较高，单次请求耗时在 500-750ms 内的接口、单次请求耗时很高，接口计算量较大的，请求耗时在 750-1000ms 的接口。在进行测试时会逐渐增加测试用例的并发量，以测试远程通信组件在不同并发度场景下的平均响应时间表现。

### 4.3.3 实验结果分析

对于功能性测试，上述实验的接口响应如表 4.4 所示，可以看出应用本服务通信组件进行远程通信的响应结果与预期的响应结果相一致。

表 4.4 服务通信组件测试结果

| 测试条目                                                                           | 响应结果                                  | 是否正确响应 |
|--------------------------------------------------------------------------------|---------------------------------------|--------|
| 未携带数据的远程服务调用<br>GET<br>http://istio-cim:10007/cim/getStationIds                | 返回了 308 条抄表区段信息                       | 是      |
| 携带了数据的远程服务调用<br>GET<br>http://istio-billing:10008/billing/getBillIds/{userNum} | 以数据库中某用户为测试用例，成功返回了该用户名下 3 个用电户的用电量信息 | 是      |

接下来是通信组件的非功能性测试，不同并发量下的响应时间测试结果如表 4.5 所示。

表 4.5 服务通信组件测试结果

| 请求并发量 | 接口   | 平均响应时间 (ms) |             | 响应时间差(ms) |
|-------|------|-------------|-------------|-----------|
|       |      | OpenFeign   | Istio Feign |           |
| 50    | 接口 1 | 132.13      | 130.21      | -1.92     |
|       | 接口 2 | 379.21      | 382.15      | 2.94      |
|       | 接口 3 | 550.32      | 547.56      | -2.76     |
|       | 接口 4 | 831.75      | 829.21      | -2.54     |
| 平均值   |      |             |             | -1.07     |
| 100   | 接口 1 | 132.51      | 131.32      | -1.19     |
|       | 接口 2 | 380.56      | 381.22      | -1.19     |
|       | 接口 3 | 548.32      | 547.66      | -0.66     |
|       | 接口 4 | 835.41      | 831.56      | -3.85     |
| 平均值   |      |             |             | -1.72     |
| 500   | 接口 1 | 148.22      | 153.54      | 5.32      |
|       | 接口 2 | 450.45      | 440.98      | -9.47     |
|       | 接口 3 | 590.78      | 575.72      | -15.06    |
|       | 接口 4 | 891.71      | 870.13      | -21.58    |
| 平均值   |      |             |             | -10.20    |
| 1000  | 接口 1 | 332.31      | 310.22      | -22.09    |
|       | 接口 2 | 650.41      | 621.37      | -29.04    |
|       | 接口 3 | 889.23      | 855.32      | -33.91    |
|       | 接口 4 | 1134.32     | 1091.75     | -42.57    |
| 平均值   |      |             |             | -31.90    |
| 2000  | 接口 1 | 473.71      | 390.32      | -83.39    |
|       | 接口 2 | 787.87      | 703.34      | -84.53    |
|       | 接口 3 | 1212.32     | 1109.23     | -103.09   |
|       | 接口 4 | 1543.61     | 1420.31     | -123.30   |
| 平均值   |      |             |             | -98.57    |

从表 4.5 中可以看出，随着请求并发量的增加，接口的响应时间也会逐渐减慢，这一点两种远程通信组件是一致的，当并发量达到 1000 的时候，两种通信组件的接口响



应时间明显下降,在并发量不足 500 时,两种通信组件在响应时间上差别不大,当并发量高于 500 时,本文实现的远程通信组件要优于 OpenFeign,有 3%~6%的提升。经过实验验证本文实现的服务通信组件可以替代 OpenFeign 成为服务网格架构下电力营销系统的远程通信解决方案。

本组件除了在响应时间上要优于开源的远程通信组件,最终的组件大小也要优于开源的远程通信组件,如表 4.6 所示,本组件相较于 OpenFeign 大小减小了 4.9%,在不影响性能的情况下更轻量化。

表 4.6 服务通信组件测试结果

| 组件名  | OpenFeign(KB) | IstioFeign(KB) | 轻量化程度 |
|------|---------------|----------------|-------|
| 组件大小 | 204           | 194            | 4.9%  |

## 4.4 MMRSA 算法资源调度能力测试

### 4.4.1 实验目标

本节实验的主要目的是对本文提出的基于 AHP-熵权法的多指标资源调度策略 MMRSA 的资源调度均衡性进行验证和分析,首先部署实现了本资源调度策略的定制资源调度器,通过与默认的集群资源调度器进行对比,验证本定制资源调度器在资源调度均衡度上的效果。

### 4.4.2 实验方案

首先将实现了本文提出的 MMRSA 资源调度算法的资源调度器部署到当前的 Kubernetes 集群当中。如图 4.17 是 MMRSA 资源调度器的集群内运行状态图。

```
[root@istio-master kubernetes]# kubectl get pods -n kube-system | grep mmrsa
mmrsa-scheduler-5988c7b98b-s6r28      1/1      Running    0          7m44s
[root@istio-master kubernetes]# kubectl describe pod mmrsa-scheduler-5988c7b98b-s6r28 -n kube-system
Name:                                mmrsa-scheduler-5988c7b98b-s6r28
Namespace:                           kube-system
Priority:                              2000000000
Priority Class Name:                   system-cluster-critical
Node:                                 istio-node2/172.16.0.26
Start Time:                           Thu, 28 Mar 2024 10:46:00 +0800
Labels:                               component=mmrsa-scheduler
                                      pod-template-hash=5988c7b98b
Annotations:                          cni.projectcalico.org/containerID: 0fc8b5b0739d525affd93132733675258cf9f79b1f856fd1375f8fb4c20e6e2c
                                      cni.projectcalico.org/podIP: 192.168.112.18/32
                                      cni.projectcalico.org/podIPs: 192.168.112.18/32
Status:                               Running
IP:                                   192.168.112.18
IPs:                                  IP: 192.168.112.18
Controlled By:                        ReplicaSet/mmrsa-scheduler-5988c7b98b
```

图 4.17 MMRSA 资源调度器集群内运行状态图

其次是验证本资源调度器调度均衡度上的效果,本实验采用对照试验的方式,设置一个对照组为默认的资源调度器,实验组为应用了本文提出的 MMRSA 算法的定制资源调度器,其他资源情况都与实验组保持一致。



为了模拟实际部署中的资源调度场景，本实验将部署电力营销系统当中的 5 个微服务模块各 10 个，这 5 个微服务如表 4.7 所示，一共 50 个 pod，对 50 个 pod 进行逐一部署，每 5 个 pod 为一组，每组 5 个微服务的 pod 各一个，部署成功后采集各个服务器的资源状态并计算集群的服务器资源利用率，直到 50 个 pod 全部部署完毕。采集的信息会包含本定制资源调度器考虑的四个维度的指标信息：CPU 利用率、内存利用率、网络带宽利用率以及磁盘 I/O 利用率。通过这些信息可以计算出对应的资源利用率均衡度，进一步对实验结果进行数据分析。

表 4.7 实验使用的微服务列表

| 微服务模块 | cim | billing | cfs | flowable | ams |
|-------|-----|---------|-----|----------|-----|
|-------|-----|---------|-----|----------|-----|

#### 4.4.3 评价指标

为了衡量资源调度器的资源调度均衡度，本文构造了一个评价函数来计算该均衡度。首先本文关注的服务器资源一共有四个维度，CPU、内存、网络带宽和磁盘 I/O，所以评价函数会基于这四个维度的资源利用率来计算，负载均衡度  $M$  的计算公式如式（4-1）所示：

$$M = \sum_p \frac{1}{N} \sum_{i=1}^N |U_i^p - U_{avg}^p| \quad (4-1)$$

式中： $P$  代表不同的资源类型，也就是上述的四个维度的服务器资源指标， $N$  为服务器集群中节点的个数， $U_i^p$  代表服务器节点  $i$  上资源类型  $P$  的资源利用率， $U_{avg}^p$  代表服务器集群中资源类型  $P$  的平均利用率，该平均利用率的计算公式如式（4-2）所示：

$$U_{avg}^p = \frac{1}{N} \sum_{i=1}^N U_i^p \quad (4-2)$$

资源均衡度  $M$  数值越大，代表服务器的资源均衡度越差，反之， $M$  数值越小，代表当前 Kubernetes 集群中的资源均衡度越好。

#### 4.4.4 实验结果分析

基于 Prometheus 采集到的资源利用率数据，通过式（4-1）和式（4-2）可以计算出对应的资源均衡度  $M$ ，图 4.18 是本节定义的评价函数计算出的资源均衡度  $M$  的变化折线图，表 4.8 是对应的资源调度均衡度  $M$  的数据以及对应的优化程度。从折线图中，可以看出 MMRSA 调度算法和默认的资源调度算法随着 pod 部署数量的不断增长，资源均衡度都会平稳下降，这也证明了两种资源调度算法调节资源调度均衡度方面的能力，表 4.8 计算得到本文提出的 MMRSA 资源调度算法在部署相同数量 pod 的情况下平均要比默认的资源调度算法的资源调度均衡度提高 11.3%，表明 MMRSA 算法能够

实现更为均衡的资源调度。

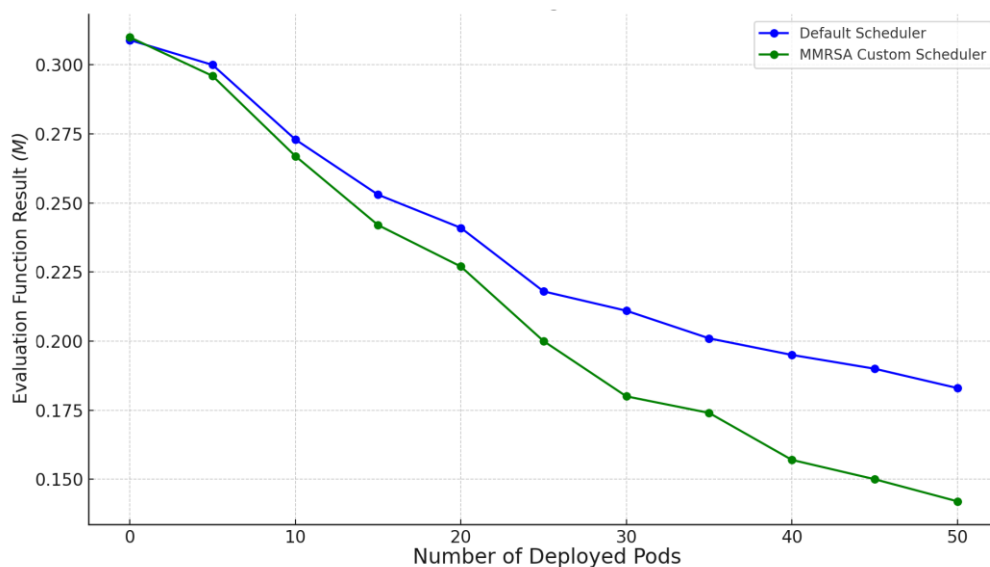


图 4.18 资源均衡度  $M$  变化折线图

以上结果支持了 MMRSA 资源调度算法在资源分配均衡性以及避免资源瓶颈方面的优势。通过在部署过程中保持较低的资源均衡度  $M$  和平稳的下降趋势，MMRSA 调度器不仅可以提高资源的调度均衡性，还可以保证应用的稳定性和可靠性。

表 4.8 服务通信组件测试结果

| 当前部署节点个数 | 资源调度均衡度 $M$ |       | 优化程度  |
|----------|-------------|-------|-------|
|          | Default     | MMRSA |       |
| 5        | 0.300       | 0.296 | 1.3%  |
| 10       | 0.273       | 0.267 | 2.1%  |
| 15       | 0.253       | 0.242 | 4.3%  |
| 20       | 0.241       | 0.227 | 5.8%  |
| 25       | 0.218       | 0.200 | 9.0%  |
| 30       | 0.211       | 0.180 | 14.6% |
| 35       | 0.201       | 0.174 | 13.4% |
| 40       | 0.195       | 0.157 | 19.4% |
| 45       | 0.190       | 0.150 | 21.0% |
| 50       | 0.183       | 0.142 | 22.4% |
| 平均值      |             |       | 11.3% |

## 4.5 本章小结

本章从各业务模块轻量化程度、系统多语言可扩展性、集群监控支持、流量管理灵活性等方面对重构后的系统进行了优化分析。对第三章当中设计的服务通信组件从功能性和非功能性两方面进行了测试，经过实验验证，本通信组件满足远程通信功能性需求，并且在高并发多线程场景下的接口响应时间相较于开源框架 OpenFeign 提升

了 3%~6%。其次对定制的资源调度算法 MMRSA 进行了资源调度均衡性测试，经过实验验证，本文设计的资源调度算法平均要比默认的资源调度算法的资源调度均衡度提高了 11.3%，可以实现更加均衡的资源调度。

## 结论

本文的研究依托于实习企业的“电力营销技术支持系统”项目。通过理论研究和工程实践，将原本基于微服务架构的电力营销系统整体迁移到基于 Istio 服务网格的云原生架构当中，将业务代码和服务治理代码完全解耦，通过这种方式来解决项目中间件版本混乱、服务治理代码高度侵入业务代码等问题。为了解决服务通信组件适配问题以及在项目应用场景中资源调度不均衡的问题。本文设计了一款服务通信组件以及一种资源调度算法来优化服务治理效果。本文完成的主要工作和成果如下：

(1) 重构电力营销系统项目。本文从网关与集群流量下发治理方案和集群内服务治理方案两方面对项目进行了非侵入式的服务治理方案设计，并依照此设计方案对项目进行了重构，解决项目中间件版本混乱、服务治理代码高度侵入业务代码的问题。重构后的系统业务模块去除了额外引入的繁杂的冗余中间件。各微服务的 jar 包平均压缩了 4.5%，相较于重构前更加轻量化。系统重构后解除了 java 语言限制，实现了系统的多语言可扩展性，同时拥有更加精细的集群监控能力以及更灵活的流量管理能力。

(2) 设计实现了一款适配 Istio 的声明式服务通信组件。该组件能够更好的满足服务网格场景下的远程通信需求，去除了开源远程通信组件中的冗余 SDK，将负载均衡等任务交给服务网格来完成。完成后的模块大小比开源组件 OpenFeign 要小 4.9%。同时在高并发场景下的性能相较于 OpenFeign 提升约 3%-6%。

(3) 为了解决 Kubernetes 资源调度不合理的问题。本文提出了基于 AHP-熵权法的多指标资源调度算法 MMRSA，并集成了资源监控系统 Prometheus，使资源调度器可以感知到节点可用网络带宽以及磁盘 I/O 并依据多个维度的数据进行打分和资源调度。经实验证实，本文设计的资源调度算法平均要比默认的资源调度算法的资源调度均衡度提高了 11.3%，可以实现更加均衡的资源调度。

综上所述，本文设计的基于 Kubernetes 和 Istio 的服务治理方案能够有效的进行电力营销系统的服务治理以及流量管控，并且部分治理方案已经应用于企业。同时本文设计的服务通信组件以及定制资源调度器经过实验验证都可以解决企业中遇到的实际问题，具有一定的实际应用价值。

虽然本文对项目的重构、实现的服务通信组件以及定制资源调度器在实验验证中已经得到了良好的结果，但是在实际应用中，还有一些需要进一步改进和完善的地方：

(1) 本文设计的资源调度器只考虑了当前收集到的资源数据，将这些数据作为打

分依据，后续可以考虑将服务器资源的长期数据也作为打分的参考依据进行调度算法的优化。

（2）本文设计的服务通信组件因为底层框架的原因目前只支持 HTTP 协议远程调用，后续可以考虑支持其他种类的传输协议，例如 gRPC。

（3）项目重构为服务网格架构后，更多的服务治理任务会从模块中迁移出来变成声明式的服务治理方式。目前还缺乏一个可视化的平台来对声明式的服务治理代码进行统一管理，后续可以考虑设计出这个治理配置管理系统。

## 参考文献

- [1] 廖艳艳.面向 Istio 服务网格的微服务治理平台的设计与实现[D].南京大学,2019:5-6.
- [2] 王义.微服务架构特点、技术趋势及在行业应用中关键问题研究[J].软件,2020,41(06):132-136.
- [3] 陶志,向忠清.微服务架构 Service Mesh 的设计与应用[J].自动化技术与应用,2020,39(01):49-53.
- [4] 孙杰平.论 Service Mesh 在微服务架构中的优势[J].通信世界,2018,(23):49.
- [5] BENTALEB O, BELLOUM A S Z, SEBAA A, et al. Containerization technologies: Taxonomies, applications and challenges[J]. The Journal of Supercomputing, 2022, 78(1): 1144-1181.
- [6] 崔华,楼奕华,狄坤等.一种基于 Kubernetes 的轻量级 PaaS 微服务框架[J].现代信息科技,2024,8(02):70-74+81.
- [7] WANG L, JIANG Y X, WANG Z, et al. The operation and maintenance governance of microservices architecture systems: A systematic literature review[J]. Journal of Software: Evolution and Process, 2023, 35(10): e2433.
- [8] CALCOTE L, BUTCHER Z. Istio: Up and running: Using a service mesh to connect, secure, control, and observe[M]. O'Reilly Media, 2019: 36-38.
- [9] MENG L, SUN Y, ZHANG S. Capestor: A Service Mesh-Based Capacity Estimation Framework for Cloud Applications[C].Cloud Computing—CLOUD 2020: 13th International Conference, Held as Part of the Services Conference Federation, SCF 2020, Honolulu, HI, USA, September 18-20, 2020, Proceedings 13. Springer International Publishing, 2020: 242-249.
- [10] SAMANI F S, STADLER R. A Framework for dynamically meeting performance objectives on a service mesh[J]. arXiv preprint arXiv:2306.14178, 2023.
- [11] ALI S A, ZAFAR M W. Istio Service Mesh Deployment Pattern for On-Premises[J]. International Journal of Computer Science and Technology, 2021, 5(1): 472-505.
- [12] 张国生.基于 Kubernetes 的无服务器计算与微服务集成架构研究[J].中国电子科学研究院学报,2023,18(01):48-55.
- [13] 赵鹏,刘畅,贾智宇等.高可用环境下的服务网格部署实践[J].信息通信技

- 术,2021,15(03):42-47.
- [14] SHITOLE A S. Dynamic Load Balancing of Microservices in Kubernetes Clusters Using Service Mesh[D]. National College of Ireland, 2022:30-34.
- [15] SHEIK O, DIKALEH S, MISTRY D, et al. Modernize digital applications with microservices management using the istio service mesh[C].Proceedings of the 28th Annual International Conference on Computer Science and Software Engineering. 2018: 359-360.
- [16] YUN Q, CHEN P, TAN D. Research on Elastic Cloud Resource Management Strategies based on Kubernetes[C].2022 IEEE International Conference on Advances in Electrical Engineering and Computer Applications (AEECA). IEEE, 2022: 441-448.
- [17] 吴封斌,李笑瑜,蒲睿强等.Istio: 微服务架构服务治理升级研究[J].网络安全技术与应用,2022,10(07):6-7.
- [18] TALEB T, BOUDI A, ROSA L, et al. Toward Supporting XR Services: Architecture and Enablers[J]. IEEE Internet of Things Journal, 2022, 10(4): 3567-3586.
- [19] NATHANIEL L, PERDANA G V, HADIANA M R, et al. Istio API gateway impact to reduce microservice latency and resource usage on kubernetes[C].2023 International Seminar on Intelligent Technology and Its Applications (ISITIA). IEEE, 2023: 43-47.
- [20] 李海飞,徐政钧.服务网格中的级联故障预测方法[J].计算机应用与软件,2021,38(11):121-130.
- [21] SONG M, LIU Q, HAI H E. A mirco-service tracing system based on istio and kubernetes[C].2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS). IEEE, 2019: 613-616.
- [22] TOWNEND P, CLEMENT S, BURDETT D, et al. Improving data center efficiency through holistic scheduling in kubernetes[C].2019 IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE, 2019: 156-164.
- [23] CAI Z, BUYYA R. Inverse queuing model-based feedback control for elastic container provisioning of web systems in kubernetes[J]. IEEE Transactions on Computers, 2021, 71(2): 337-348.
- [24] ZHONG Z, BUYYA R. A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources[J]. ACM Transactions on Internet Technology (TOIT), 2020, 20(2): 1-24.
- [25] OGBUACHI M C, GORE C, REALE A,et al.Context-Aware K8S Scheduler for Real

- Time Distributed 5G Edge Computing Applications[C].2019 International Conference on Software, Telecommunications and Computer Networks (SoftCOM).2019:76-87.
- [26] SONG S, DENG L, GONG J, et al. Gaia scheduler: A kubernetes-based scheduler framework[C].2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/-IUCC/BDC-loud/SocialCom/SustainCom). IEEE, 2018: 252-259.
- [27] WEI G Z, XI L M, JIN Z Z. Research on kubernetes' resource scheduling scheme[C].Proceedings of the 8th International Conference on Communication and Network Security. 2018: 144-148.
- [28] MENOUEUR T. KCSS: Kubernetes container scheduling strategy[J]. The Journal of Supercomputing, 2021, 77(5): 4267-4293.
- [29] 曹政,王楠,李宝安等.基于微服务容器化的分布式开源社区实现[J].软件导刊,2023,22(07):85-97.
- [30] 叶朝武,徐胜超.一种基于 Docker 的容器云平台设计与部署方法[J].信息与电脑(理论版),2023,35(18):65-67+71.
- [31] 张杨.基于 Kubernetes 的容器云平台的高可用研究与实现[D].电子科技大学,2023:34-37.
- [32] SENJAB K, ABBAS S, AHMED N, et al. A survey of Kubernetes scheduling algorithms[J]. Journal of Cloud Computing, 2023, 12(1): 87.
- [33] MEDEL V, TOLOSANA-CALASANZ R, BAÑARES J Á, et al. Characterising resource management performance in Kubernetes[J]. Computers & Electrical Engineering, 2018, 68: 286-297.
- [34] 陈晨.基于 Kubernetes 的资源调度优化方法[D].北京建筑大学,2023:36-38.
- [35] CARRIÓN C. Kubernetes scheduling: Taxonomy, ongoing issues and challenges[J]. ACM Computing Surveys, 2022, 55(7): 1-37.
- [36] SHARMA V. Managing multi-cloud deployments on kubernetes with istio, prometheus and grafana[C].2022 8th International Conference on Advanced Computing and Communication Systems (ICACCS). IEEE, 2022, 1: 525-529.
- [37] 李义勇.基于微服务的高校网站治理体系的研究与实现[J].中国现代教育装备,2023,(21):19-21+37



- [38] LI W, LEMIEUX Y, GAO J, et al. Service mesh: Challenges, state of the art, and future research opportunities[C].2019 IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE, 2019: 122-1225.
- [39] CERNY T, DONAHOO M J, TRNKA M. Contextual understanding of microservice architecture: current and future directions[J]. ACM SIGAPP Applied Computing Review, 2018, 17(4): 29-45.
- [40] SIRIWARDHANA G S, DESILVA N, JAYASINGHE L S, et al. A network science-based approach for an optimal microservice governance[C].2020 2nd International Conference on Advancements in Computing (ICAC). IEEE, 2020, 1: 357-362.
- [41] PINHEIRO C, VASCONCELOS A, GUERREIRO S. Microservice architecture from enterprise architecture management perspective[C].Business Modeling and Software Design: 9th International Symposium, BMSD 2019, Lisbon, Portugal, July 1–3, 2019, Proceedings 9. Springer International Publishing, 2019: 236-245.
- [42] MARA JÖSCH R. Managing Microservices with a Service Mesh: An implementation of a service mesh with Kubernetes and Istio[D]. Royal Technical High School,2020:34-35.
- [43] TIAN Y, HAO H, XU X, et al. Research on enterprise service governance based on service mesh[C].Journal of Physics: Conference Series. IOP Publishing, 2020, 1673(1):-45-53.
- [44] 吕亚龙.微服务架构下服务发现和服务容错技术研究[D].西安电子科技大学,2022:45-47.
- [45] HASSAN S, BAHSOON R. Microservices and their design trade-offs: A self-adaptive roadmap[C].2016 IEEE International Conference on Services Computing (SCC). IEEE, 2016: 813-818.
- [46] 常凯华. 基于服务网格的微服务治理技术研究[D].北京邮电大学,2024:45.
- [47] WANG L, JIANG Y X, WANG Z, et al. The operation and maintenance governance of microservices architecture systems: A systematic literature review[J]. Journal of Software: Evolution and Process, 2023, 35(10): e2433.
- [48] HUSSAIN F, LI W, NOYE B, et al. Intelligent service mesh framework for api security and management[C].2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON). IEEE, 2019: 0735-0742.
- [49] ZHU X, SHE G, XUE B, et al. Dissecting overheads of service mesh sidecars[C].Pro-

- ceedings of the 2023 ACM Symposium on Cloud Computing. 2023: 142-157.
- [50] JOSHI A. Selecting a service mesh implementation for managing microservices[D]. Aalto University, 2023:45-46.
- [51] RAJ P, VANGA S, CHAUDHARY A. Reliable Cloud-Native Applications through Service Mesh[M]. Wiley-IEEE Press, 2023: 233-236.
- [52] 赵鑫,李铭轩.基于服务网格的微服务故障治理[J].信息技术,2021,15(04):50-58.